

第 5 章

大且快：利用記憶體階層

5.1 介紹

5.2 記憶體技術

5.3 快取的基礎

5.4 衡量和改善快取效能

5.5 可靠的記憶體階層

5.6 虛擬機器

5.7 虛擬記憶體

5.8 記憶體階層的常用架構

5.9 以有限狀態機來控制簡單快取

5.10 平行性與記憶體階層：快取一致性

5.11 平行性與記憶體階層：冗餘廉價磁碟陣列

5.12 進階教材：實作快取控制器

5.13 實例：ARM Cortex-A8 與 Intel Core i7 的記憶體階層

5.14 走得更快：快取區塊分割與矩陣乘法

5.15 謬誤與陷阱

5.16 總結評論

5.17 歷史回顧與進一步閱讀

5.1 介紹

- ▣ 記憶體階層為程式設計者創建出記憶體又大又快的假象
- ▣ 程式執行時在同一時刻，對所有程式碼和資料的存取機率不會相同
- ▣ 區域性原則有兩種：
 - 時間區域性(temporal locality)：一筆資料若被存取，則可能很快地再次被存取
 - 空間區域性(spatial locality)：一筆資料若被存取，則其附近的資料也有即將被存取的傾向

5.1 介紹

- 程式的區域性由簡單和自然的程式結構產生
 - 迴圈顯示了高度的時間區域性
 - 指令通常是被循序地存取；循序地存取一個陣列或記錄的各個元素：自然地會具有高度的空間區域性
- 電腦記憶體實作成一個記憶體階層，來利用區域性原則的優點
 - 一個記憶體階層(memory hierarchy)包括多層不同速度和容量的記憶體
 - 圖5.1目的是呈現給使用者最便宜技術可提供的儘量多的記憶體容量，同時可提供最快的記憶體的存取速度

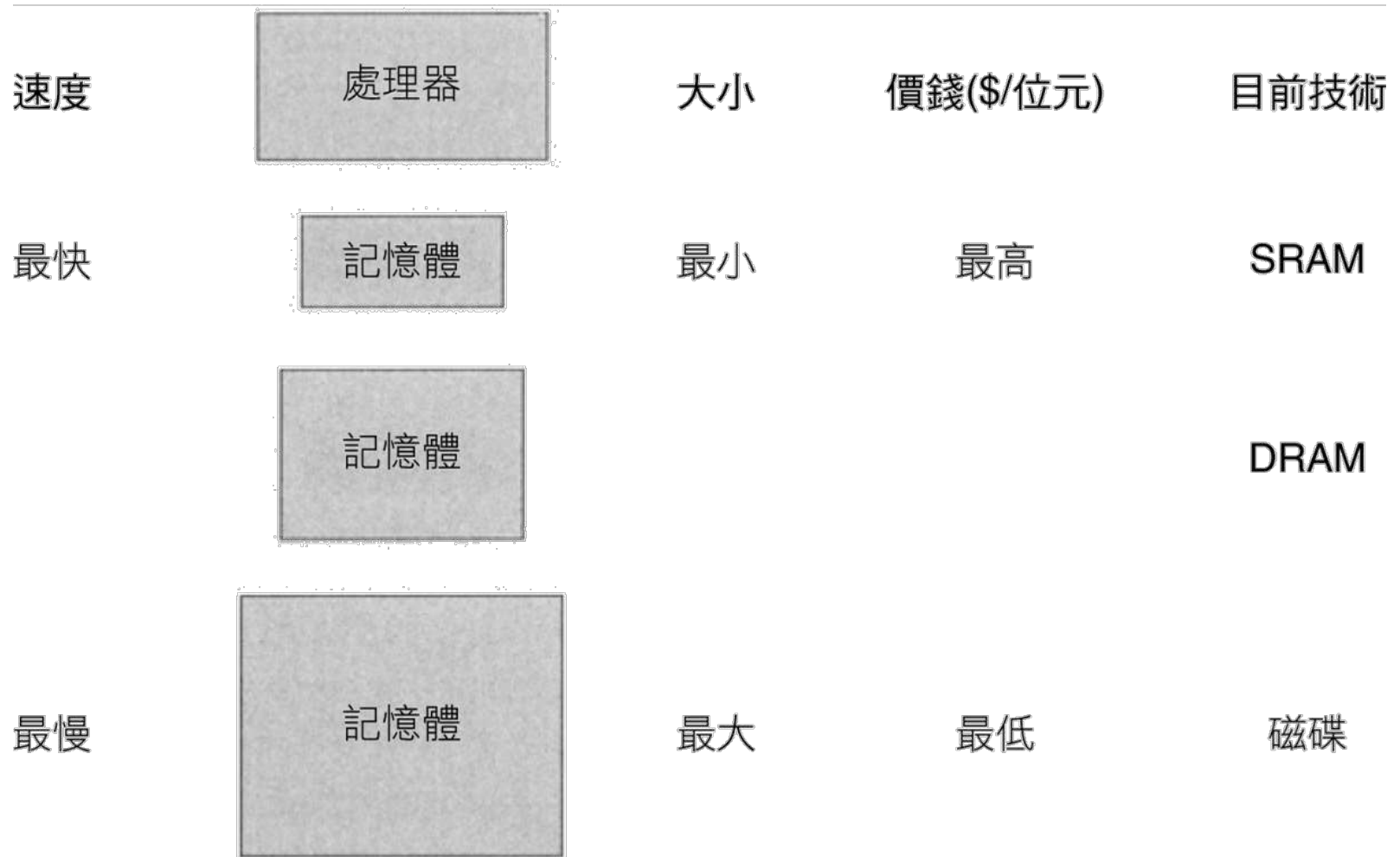


圖5.1 記憶體階層的基本結構

藉由以階層實現記憶體系統，使用者有了記憶體的假象，容量與階層中最大那層的容量一樣大，但存取速度又與最快速記憶體一樣快。在許多嵌入式系統中，快閃記憶體 (flash memory) 已經取代硬碟，且在桌上型或伺服器電腦中，可能造成新的儲存階層；詳見5.2 節

5.1 介紹

- ▣ 資料的放置同樣是階層性的：
 - 一般而言，一個較靠近處理器的層級是任一個更遠層級的子集合，而且所有的資料都放在最低的層級
 - 每次資料只在相鄰的二個層級之間複製
 - 圖5.2
 - 層級中，資訊的最小單位稱為一個區塊(block)或一行(line)

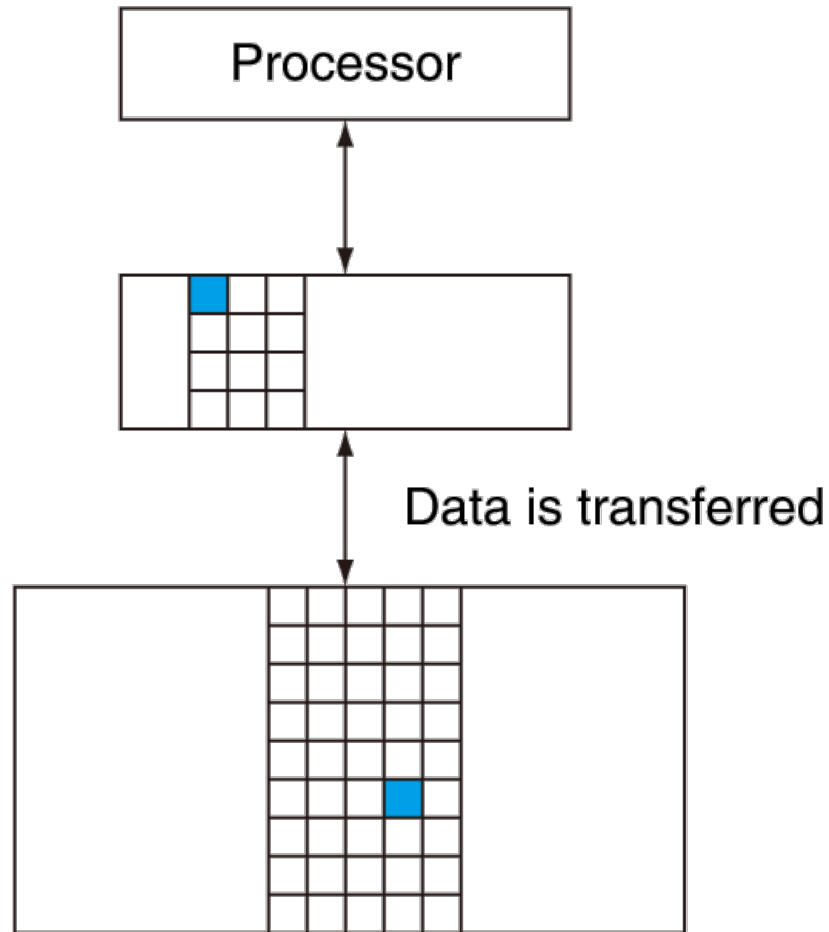


圖5.2 記憶體階層中兩兩之間都可想成有著上一層或下一層

在一層中，存在與否的資訊單位叫做區塊或行。當我們在階層間複製什麼，都以整個區塊傳遞

5.1 介紹

- 命中(hit)、錯失(miss)、命中率(hit rate)、錯失率(miss rate)、命中時間(hit time)、錯失懲罰(miss penalty)時間
 - miss penalty：用低層級中相應區塊來取代高層級中的區塊所花的時間，再加上傳送這個區塊到處理器的時間
- 用來建立記憶體系統的觀念影響著計算機的許多其他方面
 - 包括作業系統如何管理記憶體和輸出輸入，編譯器如何產生程式碼，甚至應用程式如何使用電腦

5.1 介紹

■ 記憶體階層(圖5.3)

- 將許多最近存取過的資料擺在處理器附近，以利用時間區域性來取得好處
- 藉由搬移記憶體中多個連續字組形成的區塊到階層的較上層，以利用空間區域性來得到好處
- 若命中率夠高，記憶體階層的有效存取時間接近其最高層的速度(且最快)且容量也相等於最低層的大小(且最大)

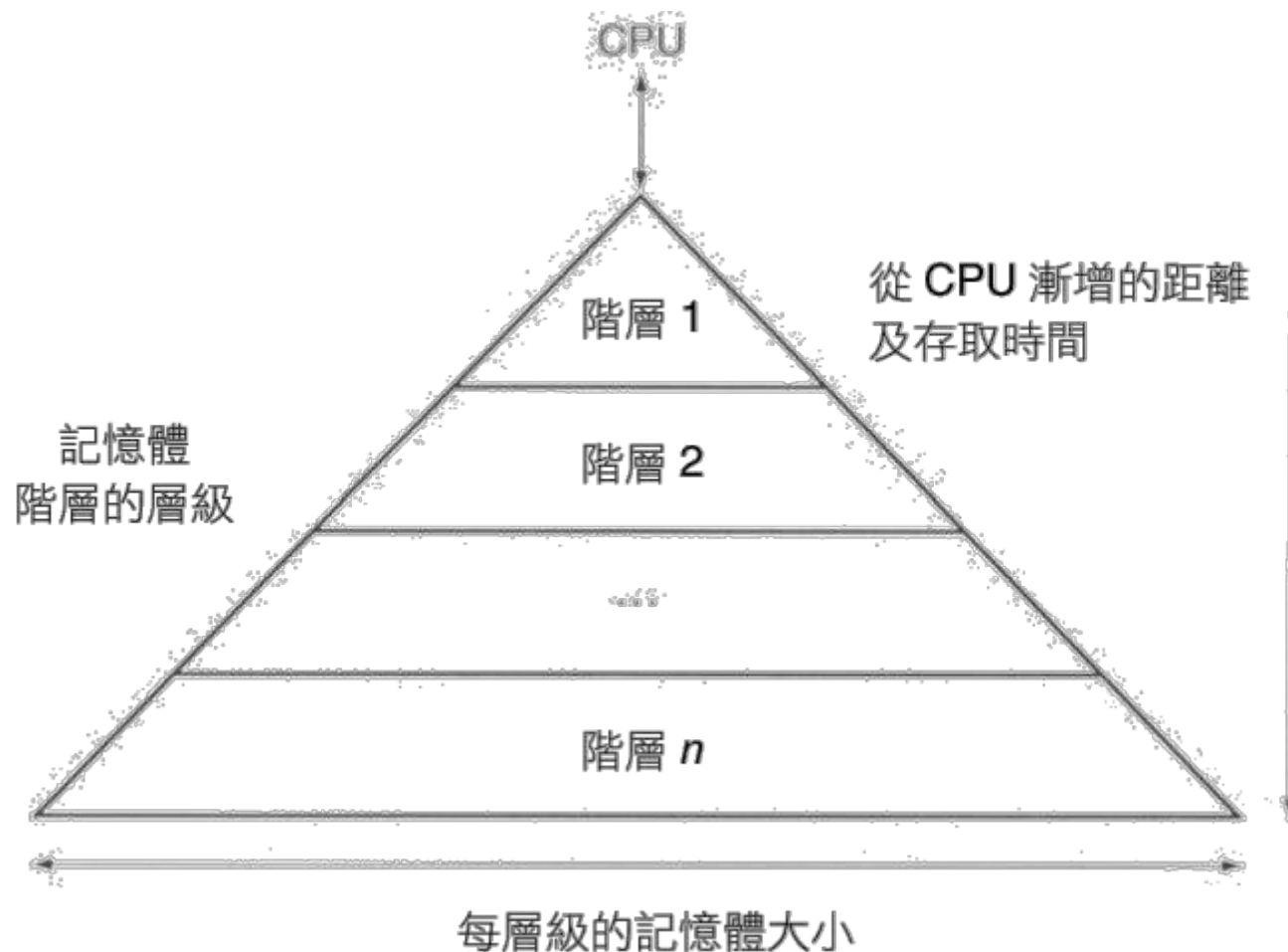


圖5.3 此簡圖顯示記憶體階層的結構：隨著與處理器距離的增加，大小也增加了。有了適當的運作機制，此結構允許處理器的存取時間主要由階層的第一層決定，也仍然有著與第 n 層記憶體一樣大的容量。維持這個假象是這章的主題。雖然本地硬碟通常是階層的底層，有些系統以區域網上的磁帶或檔案系統做為階層的下一層級。

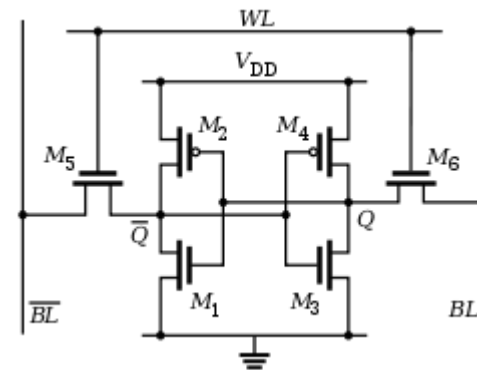
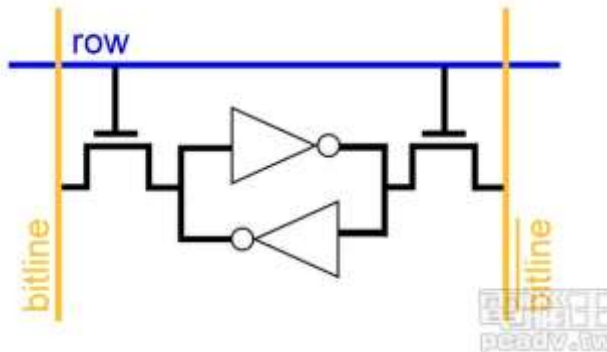
5.2 記憶體技術

- 目前有四種用於記憶體階層中的主要技術2012 年典型值：

記憶體技術	典型的存取時間	2012 年每 GiB \$
SRAM 半導體記憶體	0.5-2.5 ns	\$500-\$1000
DRAM 半導體記憶體	50-70 ns	\$10-\$20
快閃半導體記憶體	5,000-50,000 ns	\$0.75-\$1.00
磁碟	5,000,000-20,000,000 ns	\$0.05-\$0.10

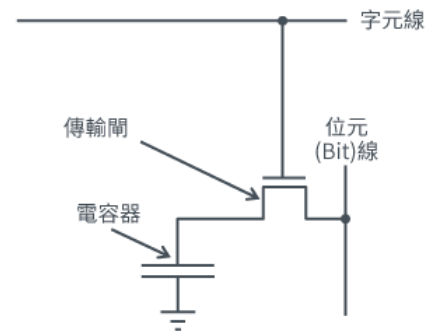
SRAM 技術

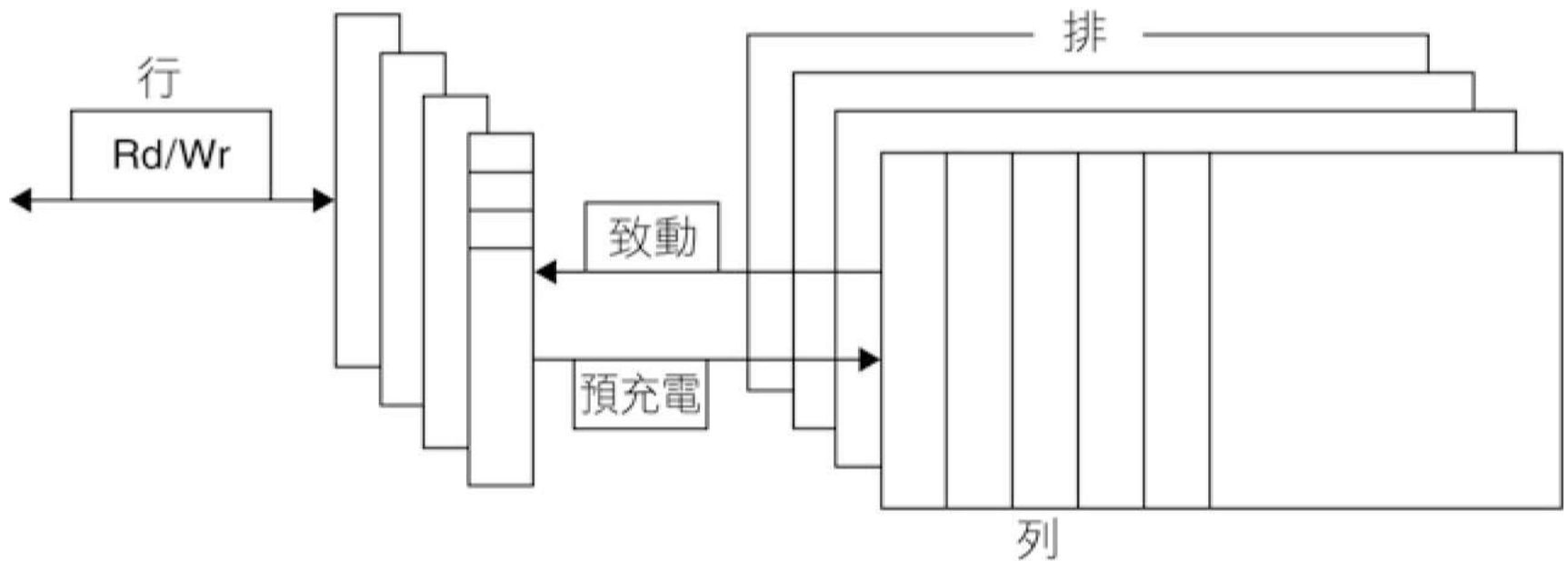
- ▣ 通常具有可提供讀取或寫入的單一存取埠
- ▣ 不需重新恢復資料(refresh) 故而其存取(access) 時間非常接近其週期(cycle) 時間
 - SRAMs 每位元一般使用六至八個電晶體來避免讀取時資料受到干擾



DRAM 技術

- 資訊在位元細胞中以電荷的形式儲存於電容中
- 一個電晶體用來存取儲存的電荷
 - 每位元遠比SRAMs 小而且便宜
- 必需週期性地重新恢復在電容上儲存電荷
 - 為何這種記憶體構造稱為動態的原因
 - 僅需先讀取其值再將之寫回即可
 - 該電荷可維持若干毫秒(milliseconds)

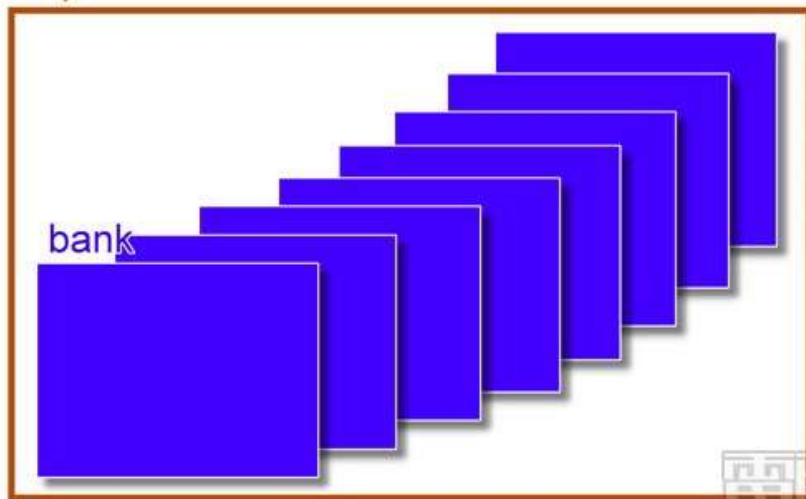




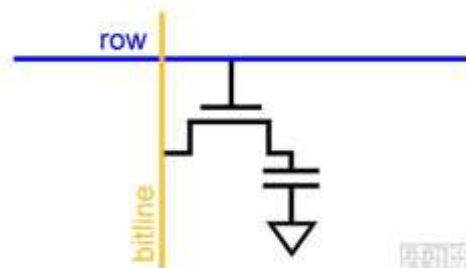
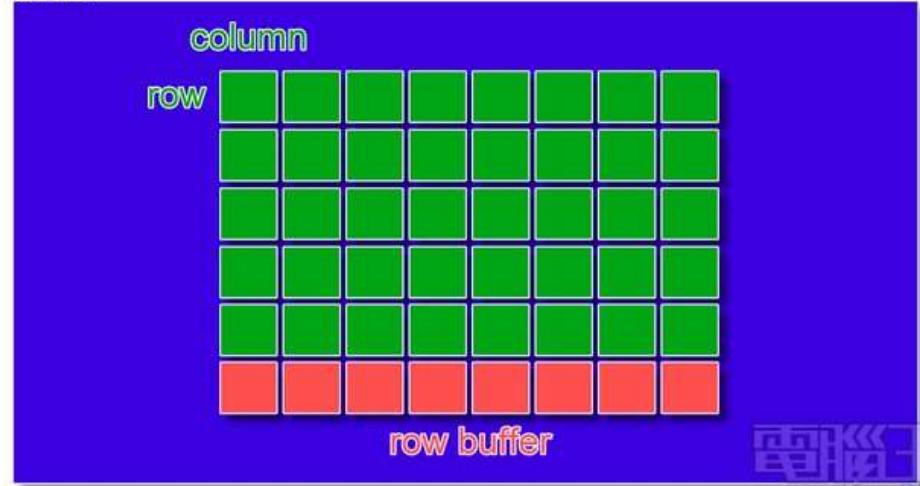
在進行下一次的 **Read** 或者 **Write** 操作前，必須要先執行 **Precharge** 操作
圖5.4 DRAM 的內部構造

新近的**DRAMs** 以記憶體排(banks)構成，一般在**DDR3** 中有四個排。每一個排中有一連串的列。送出一個**PER (precharge, 預充電)**命令可以打開或關閉一個排。一個列位址會與一個**Act (activate, 致動)**訊號一起送出，而導致對應的一個列轉移到緩衝器中。當列處於緩衝器中時，其可以以該**DRAM** 資料的寬度(在**DDR3** 中典型的是**4、8 或16** 個位元)依連續的行位址傳送數據、或以指定的區塊大小以及開始的位址來傳送。每一個命令、以及區塊的轉移，都以一個時脈作同步

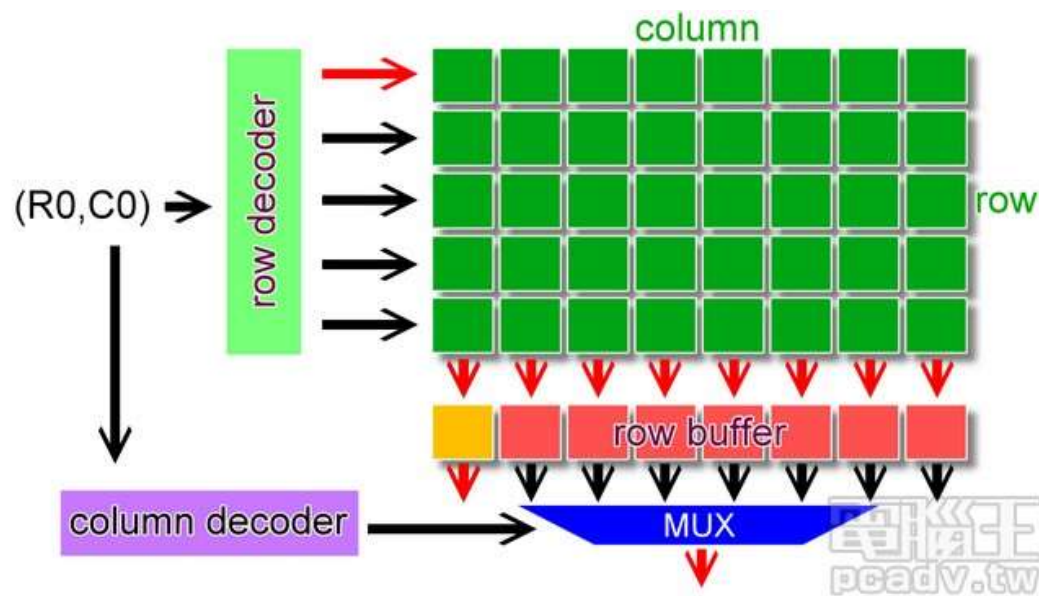
chip



bank



column (bitline)



推出年度	晶片大小	每 GiB \$	到新的列/行總存取時間	在既存列中平均行存取時間
1980	64 Kibibit	\$1,500,000	250 ns	150 ns
1983	256 Kibibit	\$500,000	185 ns	100 ns
1985	1 Mebibit	\$200,000	135 ns	40 ns
1989	4 Mebibit	\$50,000	110 ns	40 ns
1992	16 Mebibit	\$15,000	90 ns	30 ns
1996	64 Mebibit	\$10,000	60 ns	12 ns
1998	128 Mebibit	\$4,000	60 ns	10 ns
2000	256 Mebibit	\$1,000	55 ns	7 ns
2004	512 Mebibit	\$250	50 ns	5 ns
2007	1 Gibibit	\$50	45 ns	1.25 ns
2010	2 Gibibit	\$30	40 ns	1 ns
2012	4 Gibibit	\$1	35 ns	0.8 ns

圖5.5 DRAM的容量直到1996年為止大約每三年增加為四倍，然後在那時候之後增加得慢很多

存取時間的改善相對較慢且較為連續(按：意為不以大步躍進)；而成本方面大致跟隨密度的改善，雖然它經常也受其他問題、例如供需關係的影響。每gibibyte的成本並未對通貨膨脹作調整

DRAM 技術

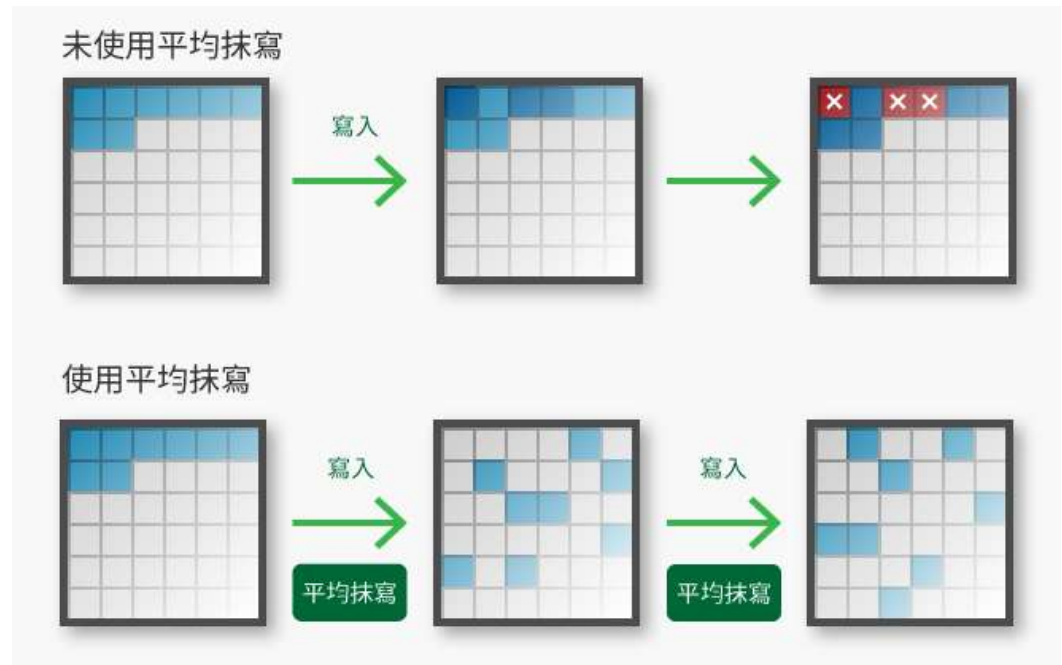
- ▣ 為了改善效能，DRAMs 暫存整個列以方便對該列的重複的存取
 - 一個緩衝器有如SRAM 般作用：不同的位元都可在此緩衝器中存取直到超出這個列的範圍為止
- ▣ 把晶片的輸出入路徑做得更寬也可改善晶片的記憶體頻寬
- ▣ SDRAMs 使用了時脈，可避免在記憶體與處理器間做同步的耗時
- ▣ 傳輸大量位元(稱為burst)時可以省去許多位址位元的使用

DRAM 技術

- ▣ 雙資料速率(Double Data Rate, DDR) SDRAM 於時脈的上升及下降緣均作傳輸
- ▣ 不祇是使用一個更快的列緩衝器，DRAM 可以在內部構建成可向多個(記憶體)排(bank)讀或寫，且每一個排都具有其自己的列緩衝器
 - ▣ 位址穿插(address interleaving)：可以讓系統對記憶體的不同bank進行同時存取
- ▣ 伺服器中使用的記憶體通常是做在小小的稱為雙排(腳位)記憶體模組(dual inline memory modules, DIMMs)的組件之上

快閃記憶體

- 快閃記憶體是 EEPROM 的特殊構造的一種型式
- 如同其他EEPROM 技術般，寫入會耗損快閃記憶體的位元
 - 耗損均攤(wear leveling：平均抹寫)

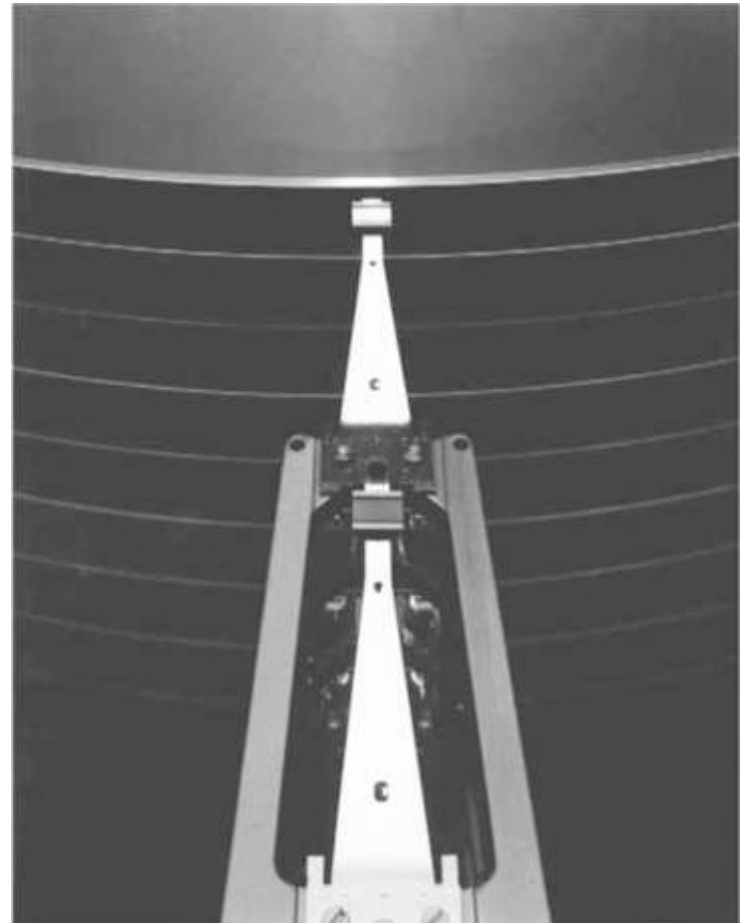


磁碟記憶體

- 如圖5.6 所示，磁性硬碟內含有一組在軸上旋轉的碟片金屬碟片的兩面佈以磁性記錄物質
 - 讀寫頭(read-write head)
 - 軌道(tracks)
 - 扇區(sectors)
 - 圓柱(cylinder)
 - 多個硬碟盤同半徑之磁軌總和

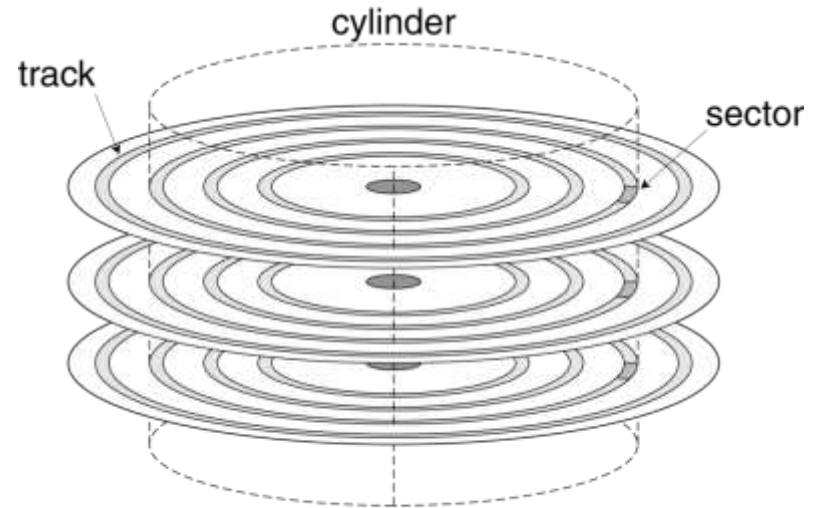
圖5.6 顯示其十個碟片以及各讀寫頭的磁碟

目前的磁碟直徑是**2.5** 或**3.5** 吋，而且今天每個驅動裝置一般具有一至二個碟片



磁碟記憶體

- ▣ 存取資料時，作業系統必需引導磁碟經過包含三個步驟的過程
 - 尋找(seek)
 - 旋轉延遲(rotational latency)
 - 傳遞時間(transfer time)

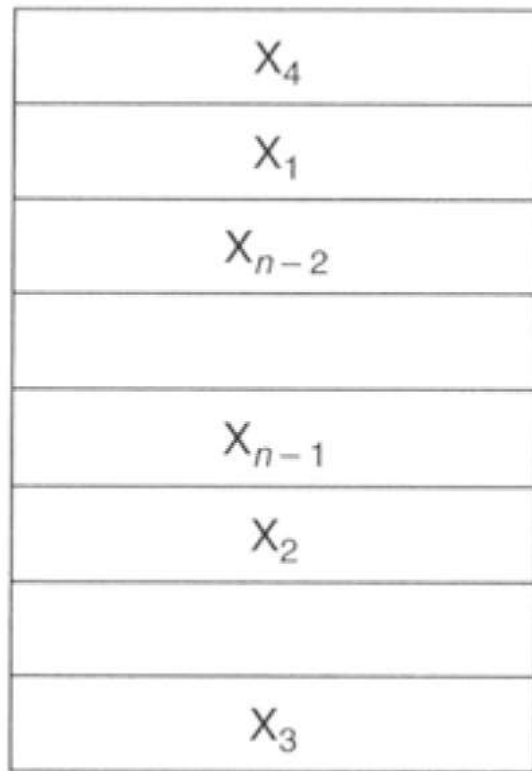


磁碟記憶體

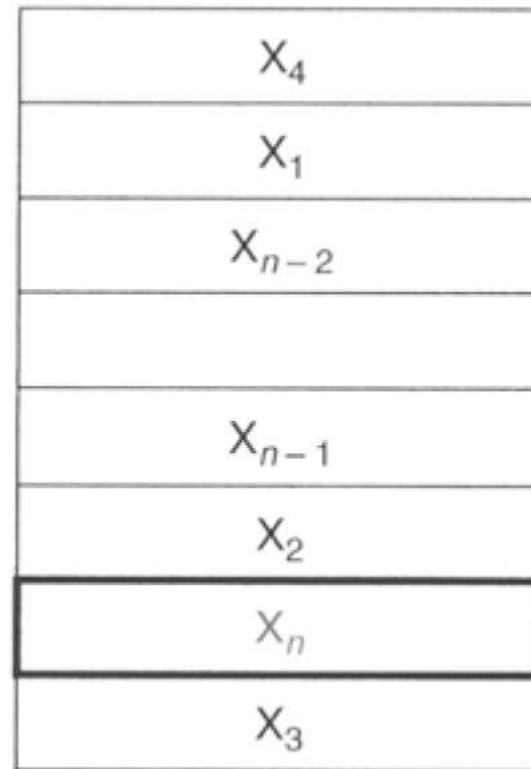
- ▣ 大部分磁碟控制器內含有內建的快取來儲存存取中經過的扇區內容
- ▣ 屬於機械裝置而較慢——快閃比它快1000 倍而DRAM快100,000 倍
- ▣ 在有限的價格下即有極高的儲存容量
- ▣ 如同快閃記憶體屬於非揮發性，而不同於快閃的是其並無寫入耗損的困擾。然而快閃遠為堅固防震

5.3 快取的基礎

- ▣ 檢視一個非常簡單的快取，此處理器的存取皆以一個字組為單位，而且快取區塊大小也只包含單一字組
 - 圖5.7
 - 如何知道資料已在快取中？
 - 若在快取中，我們如何找到此資料？



a. 存取字組 X_n 之前



b. 存取字組 X_n 之後

圖5.7 存取字組 X_n 之前與之後的快取狀態，該字組一開始不在快取中
這次存取造成一次錯失，促使快取自記憶體抓取 X_n 並放入快取中

5.3 快取的基礎

▣ 直接對映(direct mapped)

- 每一個記憶體位置直接對映到快取中的一個實際位置
 - 典型對映通常是簡單的。例如
(區塊位址)取餘數(快取中的區塊數)
 - 若快取的區塊數是2 的乘方，則取餘數時可使用位址中較低的 $\log_2$ (用區塊數表示的快取容量)個位元簡單地算出
- 圖5.8

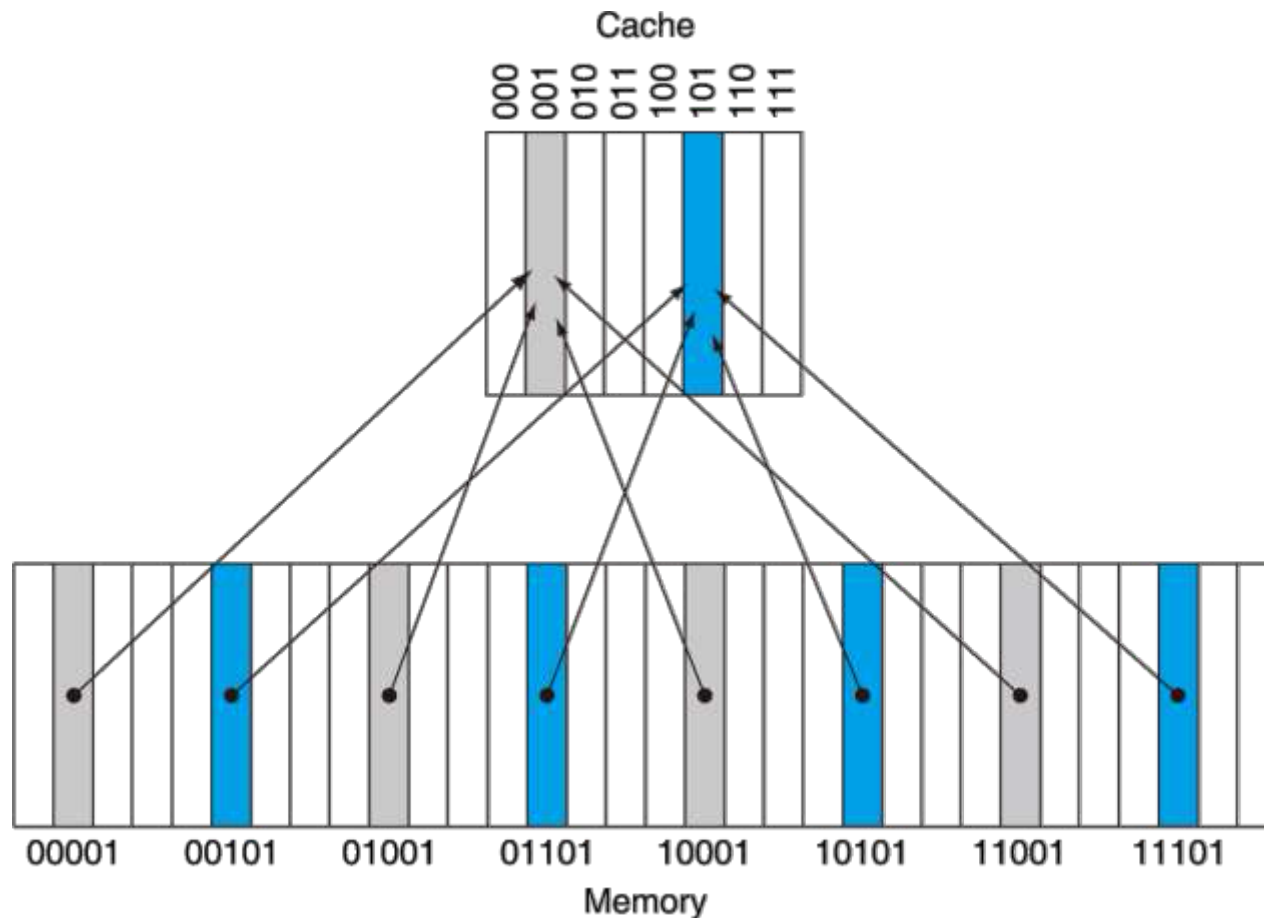


圖5.8 具有八個區塊的直接對映快取，顯示了在記憶體字組位址0 到31 之間對映到相同的快取位置

因為這快取有八字組，位址 X 對映到直接對映快取中，以 X 取8 餘數為位置的字組。也就是，以最低三位元 $\log_2(8)=3$ 為快取索引。因此，位置 00001_2 、 01001_2 、 10001_2 及 11001_2 都對映到快取區塊 001_2 ，同時，位置 00101_2 、 01101_2 、 10101_2 及 11101_2 都對映到快取區塊 101_2

5.3 快取的基礎

- 加入一組標籤(tags)，內含有所需的位址資訊，來決定快取中的字組是否為所需求的字組
 - 標籤僅需包含位址的高位元部分，沒用到的位元則當作快取的索引
 - 例如，在圖5.8 中，我們僅需要5 個位元位址中的最高2 位元做為標籤
- 使用一個有效位元(valid bit)來指出一個區塊中的項目是否包含有意義的位址

Cache Example 存取一個快取

- ▣ 8-blocks, 1 word/block, direct mapped
- ▣ Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
26	11 010	Miss	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	11	Mem[11010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
18	10 010	Miss	010

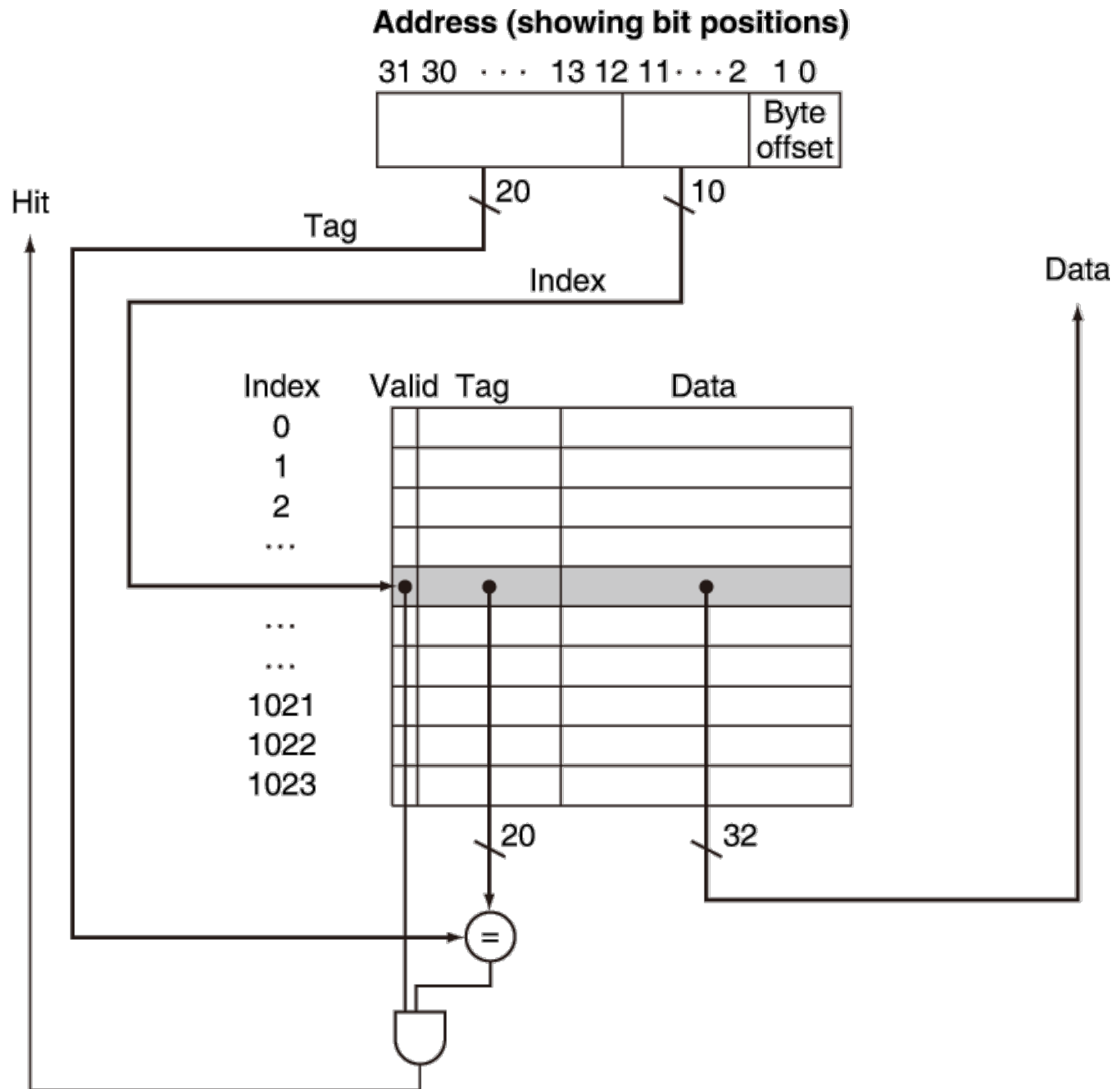
Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	10	Mem[10010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

存取一個快取

- ▣ 圖5.10表示一個存取的位址如何被區分為：
 - 標籤欄位，用來與快取中標籤欄位值做比對
 - 快取索引，用來選取區塊
 - 直接對映快取的總區塊數必須是2 的乘方
 - 在MIPS 架構，由於字組以4 個位元組對齊因此，在區塊中選取字組時，最低的2 個位元是被忽略的
 - 之前提到區塊的大小是一個字組，但正常會是多個字組

圖5.10 此快取的低位元位址用以選擇包含資料字組及標籤的一個快取區塊

這快取保存**1,024** 字組或**4 KiB** 位元組。本章中假設**32** 位元位址。從快取來的標籤與位址的高位元部份比較，以決定此快取區塊是否與請求的位址相符。因此快取有 **2^{10}** (或**1024**)字組以及區塊大小為一字組，十位元要用來索引快取，剩下 **$32-10=20$** 位元與標籤比較。如果這標籤等於位址的前二十位元，且有效位元為**on**，則這請求為快取命中，且將該字組提供給處理器。否則，即發生錯失



存取一個快取

▣ 假設：

- 32 位元的位元組位址
- 直接對映快取
- 快取容量是 2^n 個區塊，所以其索引是 n 個位元
- 區塊大小是 2^m 個字組，(2^{m+2} 個位元組)所以在區塊中的字組使用 m 個位元，兩個位元是位元組的部分
- 標籤欄位的大小是：

$$32 - (n+m+2)$$

存取一個快取

- 直接對映快取的總位元數是：

$$2^n \times (\text{區塊大小} + \text{標籤大小} + \text{有效欄位大小})$$

$$2^n \times (2^m \times 32 + (32 - n - m - 2) + 1) = 2^n \times (2^m \times 32 + 31 - n - m)$$

- 慣例上不計算標籤和有效位元的大小，只算快取中的資料容量
- 在圖5.10 中的快取是為4 KiB 的快取

存取一個快取

- 較大的區塊通常可利用空間區域性來降低錯失率
 - 如圖5.11
 - 在區塊中字組間的空間區域性隨著非常大容量的區塊而減少
 - 增加區塊容量的更嚴重問題是錯失的成本增加
 - 到階層中下一個較低層級取得該區塊的時間包含二部分：取得第一個字組的延遲時間，以及搬移區塊中其餘字組至快取的時間

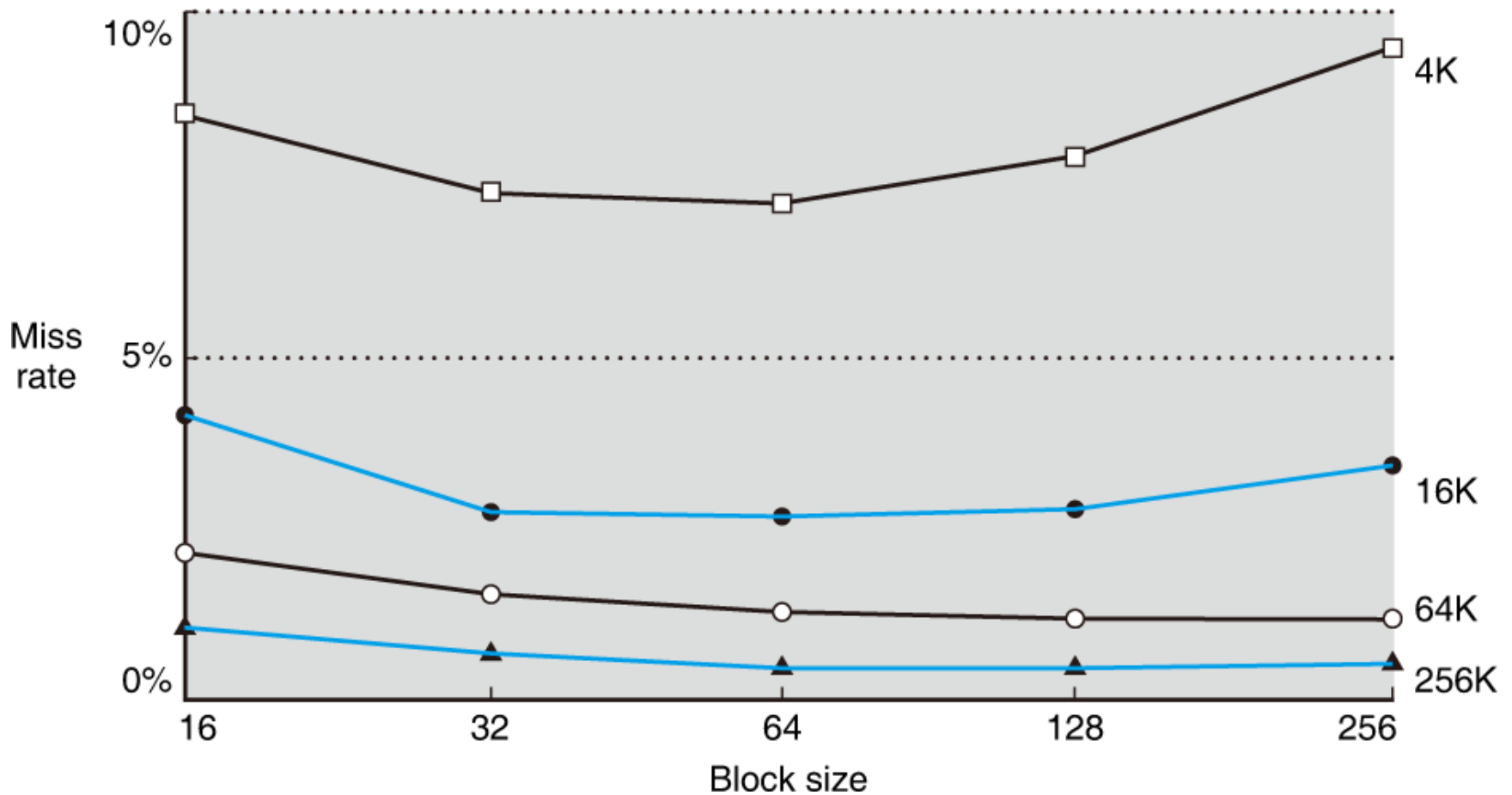


圖5.11 命中率對比區塊大小

注意錯失率其實上升了，如果區塊對比快取太大。每條線代表不同大小的快取(這圖跟關聯度無關，待會討論)。不幸地，包含區塊大小測試的SPEC CPU2000 軌跡跑太久，因此這測試資料來源基於SPEC92

處理快取錯失

- ▣ 快取的錯失處理在處理器控制單元以及另一個可啟動記憶體存取和重填快取的控制器下共同完成
- ▣ 處理快取錯失會產生管線停頓

處理快取錯失

- 在指令快取錯失時，要採取的步驟：
 - ① 送原始的PC 值(目前的PC-4)到記憶體
 - ② 指示主記憶體執行讀取
 - ③ 寫入快取區塊：把資料從記憶體放入快取記錄的資料部分，將資料位址的高位元部分(來自ALU)寫入標籤欄位，並將有效位元設為on
 - ④ 重新啟動指令執行的第一步，這將重新取得指令，這次會在快取中找到指令

處理寫入

- ▣ 假設執行一個儲存指令時，我們只將資料寫到資料快取中(而沒有寫到主記憶體)
 - 快取和記憶體是不一貫(inconsistent)的
- ▣ 要保持主記憶體和快取一貫性：寫透 (write-through)
- ▣ 另一重要考慮是寫入錯失時該怎麼辦
 - 首先從記憶體擷取包含該字組的區塊。之後將區塊放入快取中，覆蓋寫入造成快取區塊錯失的字組，寫入快取後同時使用完整的位址將字組寫到主記憶體中
 - 無法提供非常好的效能

處理寫入

- 一個方法是使用寫入緩衝器(write buffer)
 - 存放等待寫入主記憶體의資料
 - 資料寫入至快取和寫入緩衝器之後，處理器可以繼續執行
 - 當寫入主記憶體完成後，寫入緩衝器的位置可被釋出
 - 如果記憶體完成寫入的速度，比處理器產生寫的速度慢時，緩衝空間多大都沒有幫助

處理寫入

- ▣ 替代寫透方法的一個選擇是稱為寫回(write-back)或複製回(copy back)
 - 當寫入發生時，新值僅寫回到快取中的區塊
 - 當更新過的區塊要被置換時，才寫入階層中較低層的記憶體
 - 寫回可改進效能，特別是當處理器產生寫和記憶體處理寫入的速度一樣快或更快時
 - 「寫回」方法實作上是比「寫透」更複雜

- 設想在寫透式快取中發生一個錯失
 - 最常用的方法稱為寫入(錯失)配置(write allocate)
 - 另一方法是更新在記憶體中該區塊需被寫入的部分，而並不在快取中配置區塊，此法稱為寫入錯失不配置(no write allocate)
- 在寫回式快取中，若我們有一個快取錯失，且快取中被取代的區塊是被修改過的，此時必須先將區塊寫回記憶體

一個快取範例：Intrinsity FastMATH 處理器

- ▣ 圖5.12
- ▣ 處理器可在每一週期，同時存取指令字組和資料字組
 - 使用了分開的指令快取和數據快取
- ▣ 同時提供寫透和寫回方法
 - 有寫入緩衝器，其可存放一筆記錄

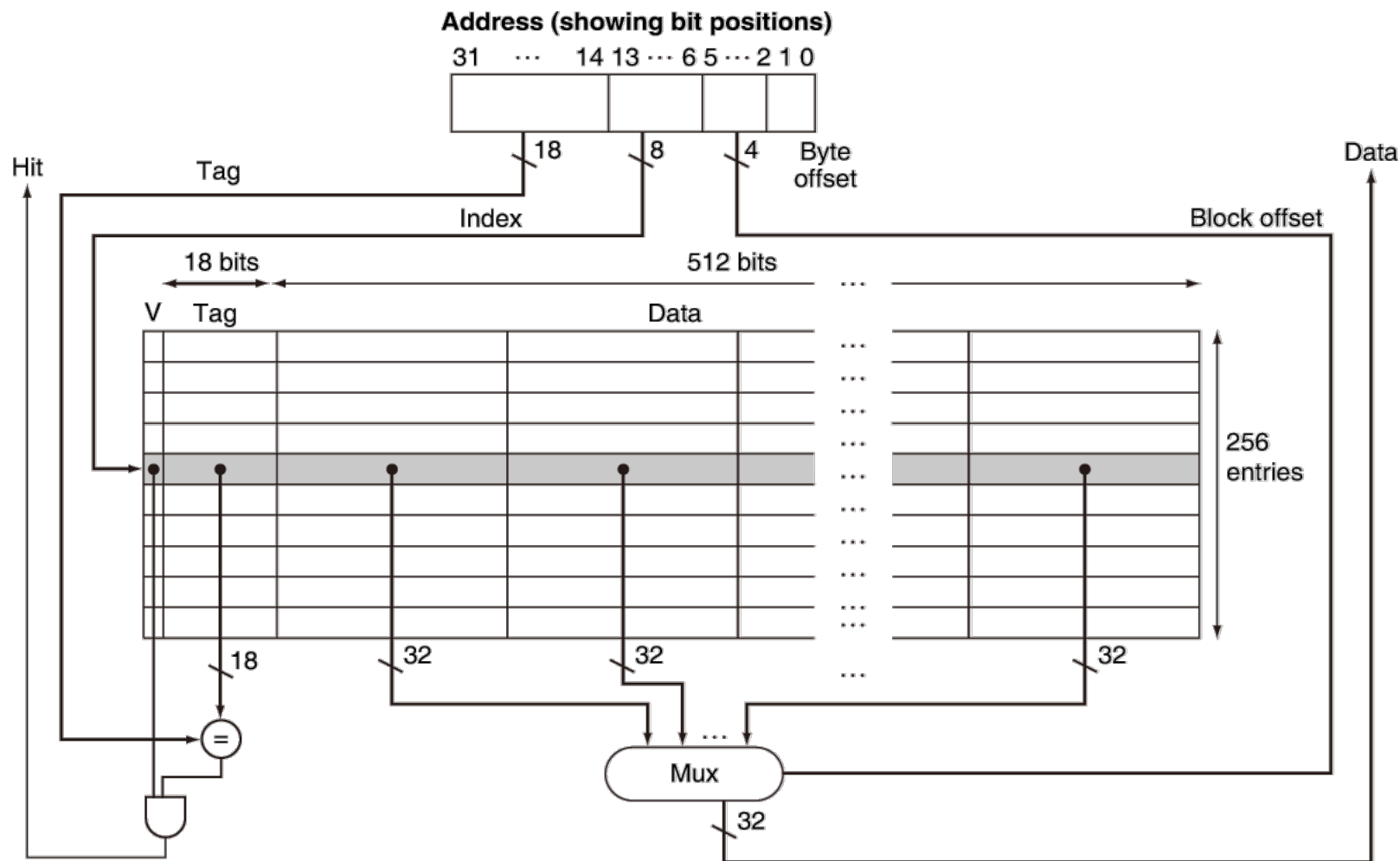


圖5.12 Intrinsity FastMATH 使用的16 KiB 快取，每個包含256 區塊，每個區塊16 字組。標籤欄18 位元寬，索引欄8 位元寬，同時用一個四位元欄(5-2 位元)以索引區塊，及以16 對1 多工器從區塊選擇字組。實際上，為了除去多工器，快取用一個分開的大RAM (隨機存取記憶體)存資料、一個小RAM存標籤以及區塊偏移提供額外的位址位元供大資料RAM 使用。在此設計，大RAM 是32 位元寬的16 倍，與快取區塊字組數量相同。

一個快取範例：Intrinsity FastMATH 處理器

■ 圖5.13 為指令快取和數據快取的錯失率

- 我們將看到記憶體錯失率和程式執行時間如何相關

指令錯失率	資料錯失率	有效合併錯失率
0.4%	11.4%	3.2%

圖5.13 Intrinsity FastMATH 執行SPEC CPU2000 測試程式時，約略的指令及數據錯失率

這合併的錯失率是**16 KiB** 指令及**16 KiB** 數據快取組合看到的有效錯失率。在考慮指令及數據存取頻率下，經由加權計算指令及數據個別錯失率而得出

- 一個容量等於指令和數據二塊分開式快取(split cache)容量總和的合併式快取(combined cache)，通常有較佳的命中率
 - 儘管如此，許多處理器使用分開的指令和數據快取，以增加快取的頻寬。(這也有較低的衝突錯失)

總結

- 使用較大的區塊可降低錯失率，並因為降低了快取中相對於資料儲存量所需的標籤儲存量，而改善快取的儲存效率，這也會增加了錯失懲罰

5.4 衡量和改善快取效能

▣ Use DRAMs for main memory

- Fixed width (e.g., 1 word)
- Connected by fixed-width clocked bus
 - Bus clock is typically slower than CPU clock

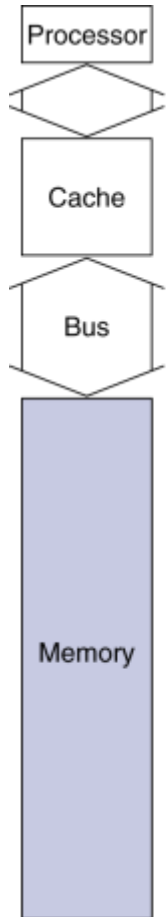
▣ Example cache block read

- 1 bus cycle for address transfer
- 15 bus cycles per DRAM access
- 1 bus cycle per data transfer

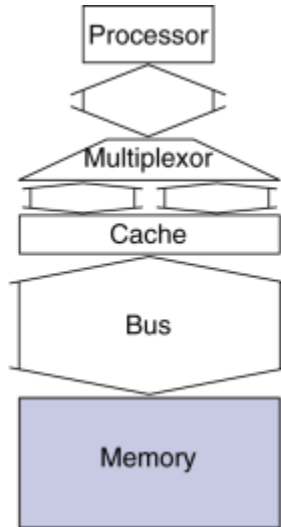
▣ For 4-word block, 1-word-wide DRAM

- Miss penalty = $1 + 4 \times 15 + 4 \times 1 = 65$ bus cycles
- Bandwidth = $16 \text{ bytes} / 65 \text{ cycles} = 0.25 \text{ B/cycle}$

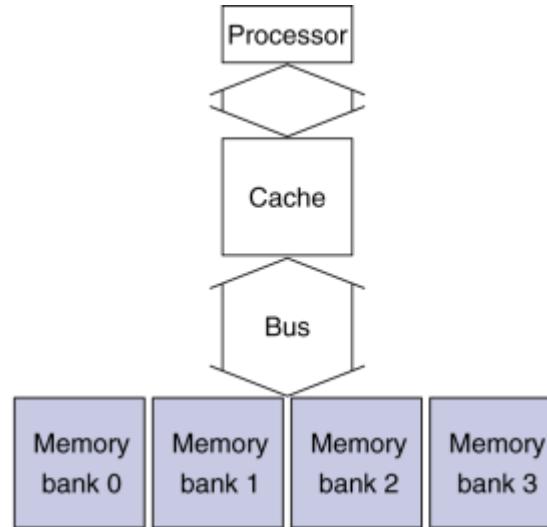
Increasing Memory Bandwidth



a. One-word-wide memory organization



b. Wider memory organization



c. Interleaved memory organization

- **4-word wide memory**
 - Miss penalty = $1 + 15 + 1 = 17$ bus cycles
 - Bandwidth = $16 \text{ bytes} / 17 \text{ cycles} = 0.94 \text{ B/cycle}$
- **4-bank interleaved memory**
 - Miss penalty = $1 + 15 + 4 \times 1 = 20$ bus cycles
 - Bandwidth = $16 \text{ bytes} / 20 \text{ cycles} = 0.8 \text{ B/cycle}$

Measuring Cache Performance

- ▣ Components of CPU time
 - Program execution cycles
 - Includes cache hit time
 - Memory stall cycles
 - Mainly from cache misses
- ▣ With simplifying assumptions:

Memory stall cycles

$$= \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

Cache Performance Example

■ Given

- I-cache miss rate = 2%
- D-cache miss rate = 4%
- Miss penalty = 100 cycles
- Base CPI (ideal cache) = 2
- Load & stores are 36% of instructions

■ Miss cycles per instruction

- I-cache: $0.02 \times 100 = 2$
- D-cache: $0.36 \times 0.04 \times 100 = 1.44$

■ Actual CPI = $2 + 2 + 1.44 = 5.44$

- Ideal CPU is $5.44/2 = 2.72$ times faster

Average Access Time

- ▣ Hit time is also important for performance
- ▣ Average memory access time (AMAT)
 - $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$
- ▣ Example
 - CPU with 1ns clock, hit time = 1 cycle, miss penalty = 20 cycles, I-cache miss rate = 5%
 - $AMAT = 1 + 0.05 \times 20 = 2\text{ns}$
 - 2 cycles per instruction

Performance Summary

- ▣ When CPU performance increased
 - Miss penalty becomes more significant
- ▣ Decreasing base CPI
 - Greater proportion of time spent on memory stalls
- ▣ Increasing clock rate
 - Memory stalls account for more CPU cycles
- ▣ Can't neglect cache behavior when evaluating system performance

以更靈活的區塊放置法來減少快取錯失

- 一個區塊可放置在快取中的任何位置的方法稱為全關聯式(fully associative)
 - 為了在全關聯式快取中找到特定的區塊，每一個快取記錄用一個比較器，所有記錄的搜尋是平行做完的
 - 顯著地增加了硬體成本，使得全關聯式放置方法只有在區塊數少的時候才可行
- 介於直接對映和全關聯式的設計稱為集合關聯式(set associative)

以更靈活的區塊放置法來減少快取錯失

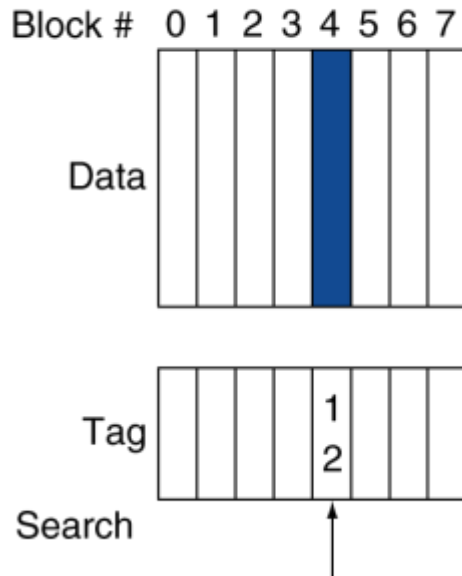
- 一個 n 路(n ways)集合關聯式快取包含一些集合，每一集合含 n 個區塊。在記憶體中的每一區塊依索引欄位對映到快取中唯一的集合，且一個區塊可被放置在該集合中的任何位置

▣ 圖5.14

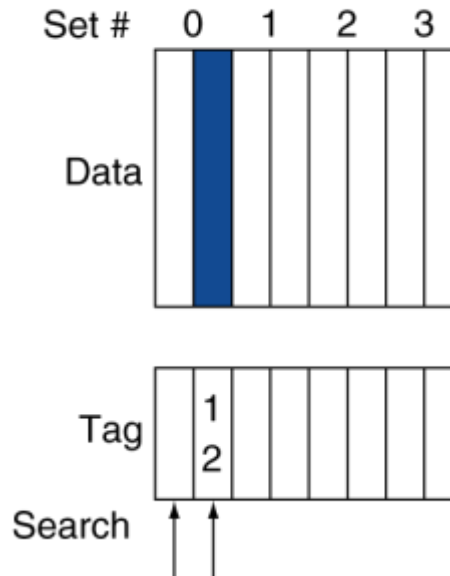
- ▣ 可以把所有區塊的放置方法視為集合關聯式的各種變型
- ▣ 增加關聯度的好處通常是減少錯失率，主要的缺點是可能增加命中時間

Associative Cache Example

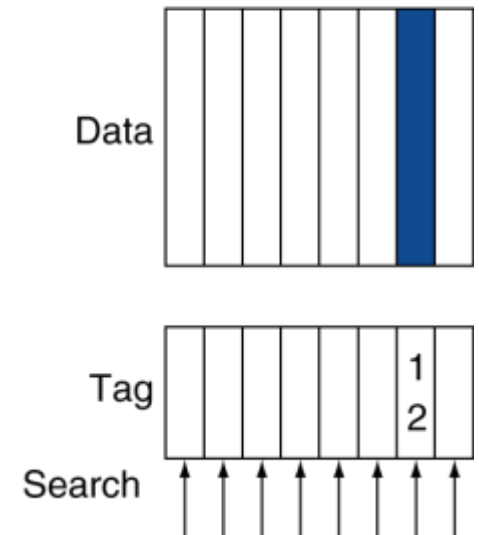
Direct mapped



Set associative



Fully associative



Associativity Example

- ▣ Compare 4-block caches
 - Direct mapped, 2-way set associative, fully associative
 - Block access sequence: 0, 8, 0, 6, 8
- ▣ Direct mapped

Block address	Cache index	Hit/miss	Cache content after access			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

Associativity Example

▣ 2-way set associative

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
0	0	miss	Mem[0]			
8	0	miss	Mem[0]	Mem[8]		
0	0	hit	Mem[0]	Mem[8]		
6	0	miss	Mem[0]	Mem[6]		
8	0	miss	Mem[8]	Mem[6]		

■ Fully associative

Block address		Hit/miss	Cache content after access			
0		miss	Mem[0]			
8		miss	Mem[0]	Mem[8]		
0		hit	Mem[0]	Mem[8]		
6		miss	Mem[0]	Mem[8]	Mem[6]	
8		hit	Mem[0]	Mem[8]	Mem[6]	

How Much Associativity

- ▣ Increased associativity decreases miss rate
 - But with diminishing returns
- ▣ Simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000
 - 1-way: 10.3%
 - 2-way: 8.6%
 - 4-way: 8.3%
 - 8-way: 8.1%

在快取中找到區塊

▣ 圖5.17 將位址分解為三段

- 其索引值用來選出含此位址的集合，在此集合中所有區塊的標籤都必須並行被搜尋
- 關聯度每增加為2 倍，索引值的長度會少1 個位元，而標籤的長度則多1 個位元
 - 在全關聯式快取中位址沒有索引欄，除了區塊偏移量之外的部分，全用來和每一個區塊的標籤做比較

標籤	索引	區塊偏移量
----	----	-------

圖5.17 集合關聯式或直接對映式快取的位址之三個部分

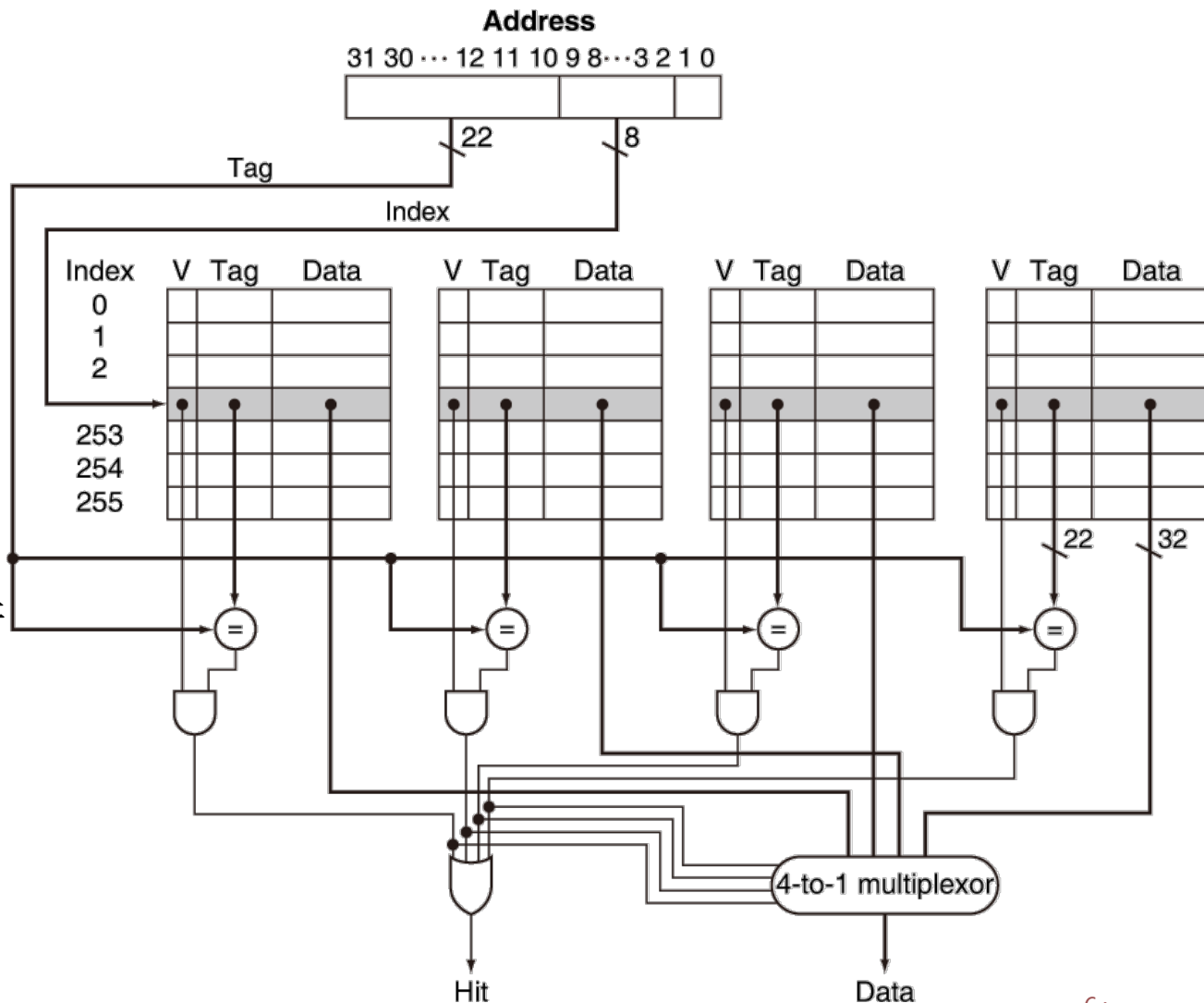
索引用以選擇集合，然後標籤用以選擇區塊，藉由與選出集合的區塊比較。區塊偏移量是區塊內所需資料的位址

在快取中找到區塊

■ 圖5.18 顯示一個4路集合關聯式快取

圖5.18 實現四路集合關聯式快取，需要四個比較器及四對一多工器

這些比較器決定選到集合(如果有)的哪些元素合於標籤。比較器輸出用以選擇資料(來自索引出集合的四個區塊之一)，以解碼過的選擇訊號控制的多工器選出。有些做法中，快取RAM的資料部分的輸出許可訊號，可用來選擇集合元素推動輸出。輸出許可訊號來自比較器，使符合的元素驅動資料輸出。這個組織方式不再需要多工器



Ex1: address 32 lines, 64kB cache, 4 ways, block size 16 words, k,n,m = ?
Total cache size = ?

address	k	n	m	2
---------	---	---	---	---

V	tag	DATA1	...	DATA _n
			...	
			...	
			...	
			...	
			...	
			...	
			...	
⋮				
			...	
			...	
			...	
			...	

V	tag	DATA1	...	DATA _n
			...	
			...	
			...	
			...	
			...	
			...	
			...	
⋮				
			...	
			...	
			...	
			...	

.....

V	tag	DATA1	...	DATA _n
			...	
			...	
			...	
			...	
			...	
			...	
			...	
⋮				
			...	
			...	
			...	
			...	

Ex2: address 32 lines, 8 ways, block size 32 words, $n = 10$, cache size ?
Total cache size = ?

address	k	n	m	2
---------	---	---	---	---

V	tag	DATA1	...	DATA _n
			...	
			...	
			...	
			...	
			...	
			...	
			...	
⋮				
			...	
			...	
			...	
			...	

V	tag	DATA1	...	DATA _n
			...	
			...	
			...	
			...	
			...	
			...	
			...	
⋮				
			...	
			...	
			...	
			...	

.....

V	tag	DATA1	...	DATA _n
			...	
			...	
			...	
			...	
			...	
			...	
			...	
⋮				
			...	
			...	
			...	
			...	

選擇置換哪個區塊

- 在集合關聯式快取中，我們可以選擇所需求的區塊要放置在何處，因此，也可以選擇要置換掉哪個區塊
 - 在全關聯式快取中，所有的區塊都是可被置換的對象
- 最常用的方法是，舉例：最久沒被用到(least recently used, LRU)
 - 當關聯度增加時，LRU 的實作變得困難

使用多層級快取減少錯失懲罰

- 大部分的處理器支援額外一層的快取
 - 第二層的快取通常在相同的晶片上
 - 存取時間遠比主憶體的存取時間小很多
- 主要快取和第二層快取二者的設計考量是有顯著差異的
 - 主要快取可專注於減少命中時間
 - 第二層快取注重錯失率
 - 主要快取通常很小，可使用較小的區塊大小，減少錯失懲罰
 - 第二層快取有較大的容量，可使用比單層快取更大的區塊，往往會使用比主要快取更高的關聯度

5.5 可靠的記憶體階層

定義故障

- 系統只會在兩種提供服務的狀態之間作轉換：
 - ① 服務達成，亦即服務可以以所規範的形式提供
 - ② 服務干擾，亦即提供的服務與所規範者不同
 - 狀態的轉換是由故障與修復造成
- 信賴度(reliability)是從某一時間點開始連續的服務達成的量度——也就是等於到發生故障的時間
 - 到發生故障的平均時間(mean time to failure, MTTF)
 - 年故障率(annual failure rate, AFR)
 - 修理的平均時間(mean time to repair, MTTR)
 - 故障間的平均時間(mean time between failures, MTBF)

5.5 可靠的記憶體階層

▣ 可用度(availability)

$$\text{可用度} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

- 每年的「可用度裡頭的九」的數目(number of "nines of availability")

▣ 組件的故障不一定就會造成系統的故障

- 以名詞錯誤(fault)來表示組件的故障
- 三種改善MTTF 的方法：
 1. 錯誤避免
 2. 錯誤容忍
 3. 錯誤預測

Hamming 單錯誤更正雙錯誤偵測(Single Error Correcting, Double Error Detecting, SEC/DED)碼

- Hamming 距離(Hamming distance)：兩個正確位元樣式之間相同位置上具有不同位元值的最少位元數目
- Hamming 使用同位碼(parity code)來做錯誤偵測
 - 當字組寫入記憶體時也同時寫入一個同位元(以1表奇同位，0表偶同位)
 - 也就是這個 $N+1$ 個位元的字組的同位應該恆為偶
 - 一個位元的同位方法可以偵測任何奇數的錯誤
- Hamming 錯誤更正碼(ECC)
 - ① 將位元由左方開始從1編號，而非如傳統的從零開始由右方編起

Hamming 單錯誤更正雙錯誤偵測(Single Error Correcting, Double Error Detecting, SEC/DED)碼

- ② 標註所有二的指數次方的位置(位置1、2、4、8、16、...)來作為放置同位位元的位置
- ③ 所有其他的位元位置(位置3、5、6、7、9、10、11、12、13、14、15、...)則作為放置資料位元之用
- ④ 同位位元的位置決定了哪些資料位元是他需要檢查的(圖5.24 以圖形表示涵蓋的範圍)，說明如下
- ⑤ 對每一群組設定同位位元，使其成為偶同位
 - 你可以從同位位元看出是否有哪一個位元出錯了
 - 可以輕易地以翻轉位元10 的值來更正錯誤
 - 如果再增加一個位元，我們可以讓碼中最小的Hamming 距離變成4
 - 可以更正單一位元的錯誤並偵測兩個位元的錯誤

- 要計算出SEC 需要多少個額外的位元，假設在 $p+d$ 個位元的字組中令 p 為同位元總數且 d 為原資料位元數
 - 可經由加入 p 個錯誤更正位元來指出一個錯誤位元的所在位置、加上一個情況來指出沒有錯誤存在：

$$2^p \geq p+d+1 \text{ 種情況，因此 } p \geq \log(p+d+1)$$

5.6 虛擬機器

- ▣ 佔有虛擬機器(VM)最早發展於1960 年代中期
- ▣ 最近受到普遍關注，因為
 - 在現代系統中隔離性與安全性日益重要
 - 標準作業系統中，安全性與可靠性的能力不足
 - 許多不相關的使用者，共享一台單一的計算機，特別是在雲端計算時
 - 幾十年來，處理器的原始速度戲劇性地增加，這使得VM 的額外負擔更可接受

5.6 虛擬機器

- 我們假設在VM上執行的指令集架構總是與硬體相同
 - 這些VM稱為(作業)系統虛擬機器(Operating System Virtual Machines)
- 單一台計算機可執行多個VM，同時可以支援多個不同的作業系統
 - 在VM中，多個作業系統需全體共享硬體資源
- 支援VM的軟體稱為虛擬機器監控器(virtual machine monitor, VMM)或超管理者(hypervisor)

5.6 虛擬機器

▣ VM 除了改進保護能力，還提供其他兩個在商業上重要的好處：

1. 管理軟體

VM 提供一個可以執行完整軟體堆疊的抽象層，甚至包括舊的作業系統，像是DOS

2. 管理硬體

VM 容許這些獨立的軟體堆疊可獨立地運作卻能共享硬體，因此降低伺服器數量的需求

虛擬機器監控器的必備條件

▣ VM 監控器必須做什麼？

- 它是客軟體的軟體介面，其必須將所有客軟體的狀態彼此相互隔離
- 它必須保護自身不受到客軟體(包含客作業系統)的侵害

▣ 質化需求是：

- 客軟體在VM的行為，除了效能相關行為或固定資源被多個VM共享的限制外，應該像它在真實硬體上執行一樣
- 客軟體應不能直接改變真實系統資源的配置

虛擬機器監控器的必備條件

- ▣ 系統虛擬機器的基本必備條件為：
 - 最少兩個處理器模式：系統和使用者
 - 僅可在系統模式中使用的特權指令子集合，其若在使用者模式中執行，則會被捕捉；所有系統資源必須只被這些特權指令控制

虛擬機器的(缺乏)指令集架構支援

- 容許VM 直接執行於硬體上的架構稱為可虛擬化
- VMM 必須確保客系統只與虛擬資源互動，因此，傳統的客作業系統是在VMM 內以使用者模式程式來執行
 - 若客作業系統經由一個特權指令嘗試存取或修改和硬體資源有關的資訊，則會被捕捉而跳到VMM 中
 - 在缺乏這種支援時，VMM 必須採取特別預防措施來辨認出所有可能造成問題的指令，並確保當它們被客作業系統執行時的行為恰當
 - 增加了VMM 的複雜性和降低執行VM 的效能

保護與指令集架構

- ▣ 保護是架構和作業系統的共同任務

5.7 虛擬記憶體

- ▣ 主記憶體可以視為輔助儲存體(通常以磁碟實作而成)的快取。這方法稱為虛擬記憶體(virtual memory)
 - 起源於二個主要的動機：
 - 使多個程式間可以有效及安全地分享記憶體
 - 消除因使用少量有限的主記憶體，而造成程式編寫上的負擔
- ▣ 虛擬記憶體和快取都是基於區域性原則
 - 虛擬記憶體使我們可以有效地共享處理器和主記憶體

5.7 虛擬記憶體

- 我們會希望將每個程式編譯成具有自己的位址獨立空間
 - 虛擬記憶體實現程式位址空間和實體位址(physical address)的轉換
- 第二個動機，是容許單一使用者程式可使用超過主記憶體大小的空間
 - 虛擬記憶體減輕程式設計者的這種困難，它自動地管理記憶體階層中的主記憶體(有時稱為實體記憶體，以有別於虛擬記憶體)和輔助記憶體的二個層級

5.7 虛擬記憶體

▣ 虛擬記憶體中

- 區塊稱為頁(page)，而虛擬記憶體錯失則稱為頁錯失(page fault)
- 處理器產生的位址，稱為虛擬位址(virtual address)
 - 其會由一個合併了硬體和軟體的機制轉換出實體位址

▣ 圖5.25

- 位址對映或位址轉換(address translation)

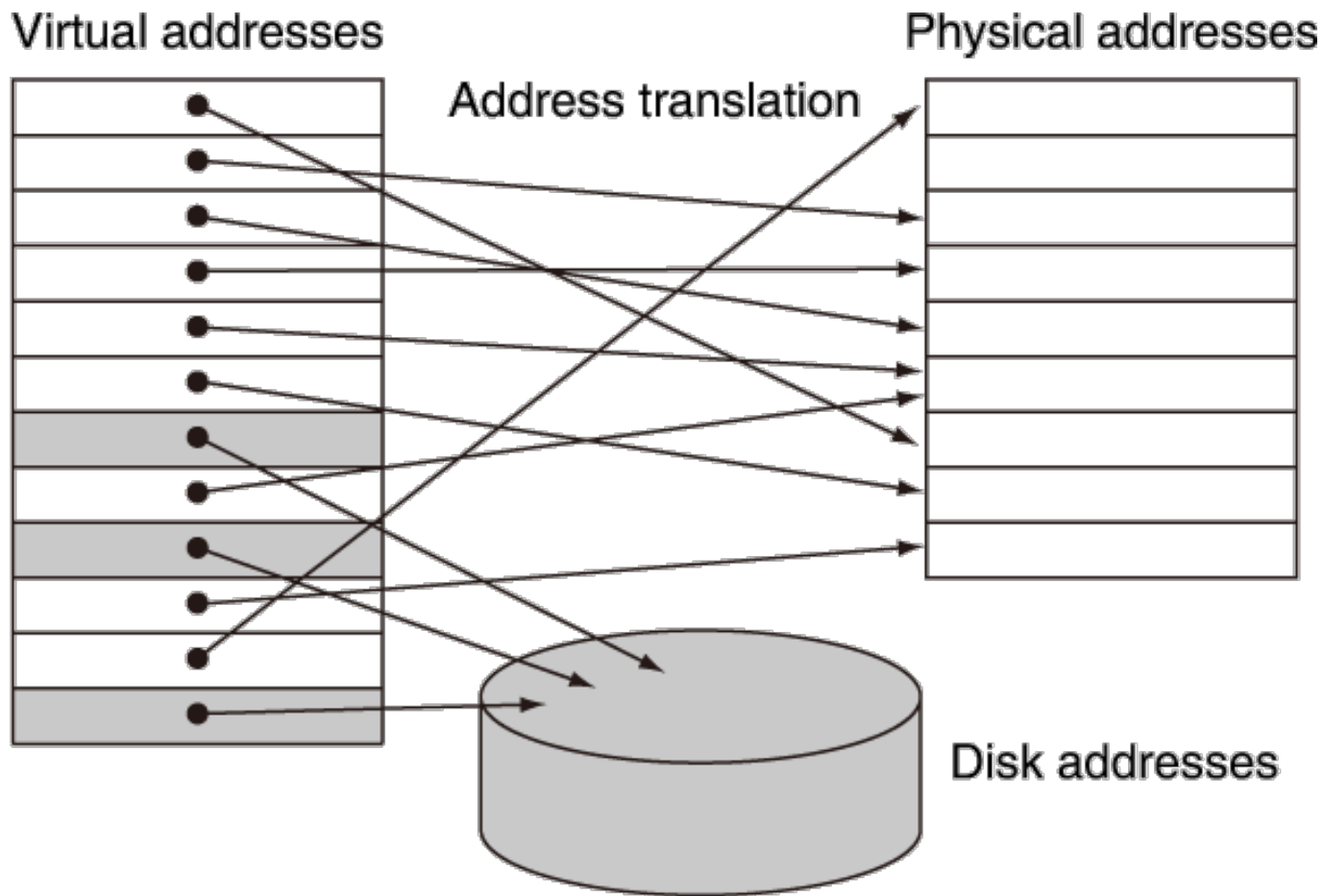


圖5.25 在虛擬記憶體中，記憶體區塊(稱為頁)從一組位址(稱為虛擬位址)被對映到另一組位址(稱為實體位址)

處理器產生虛擬位址，然而記憶體存取是使用實際位址。虛擬記憶體與實體記憶體皆被分成許多頁，使得一虛擬頁對映到一實體頁。當然，有可能某虛擬頁不在主記憶體中，因此不對映到實體位址；此情形下，該頁在磁碟中。實體頁可被二個指到相同實體位址的虛擬位址共用。這種功能可用來允許二個不同程式共用資料或程式碼

5.7 虛擬記憶體

- ▣ 虛擬記憶體管理藉由提供重新定位(relocation)來簡化程式執行時的載入工作
 - 現今使用的所有虛擬記憶體系統把程式重新定位為一組固定大小的區塊(頁)，因此不需要去找連續的記憶體區塊來放置一整個程式
 - 作業系統僅需在主記憶體中找到足夠數量的頁即可
- ▣ 在虛擬記憶體系統中，位址可分為虛擬頁編號和頁偏移量二部分
 - 圖5.26 顯示了虛擬頁編號轉換到實體頁編號
 - 不需改變的頁偏移量的位元數決定了頁的大小

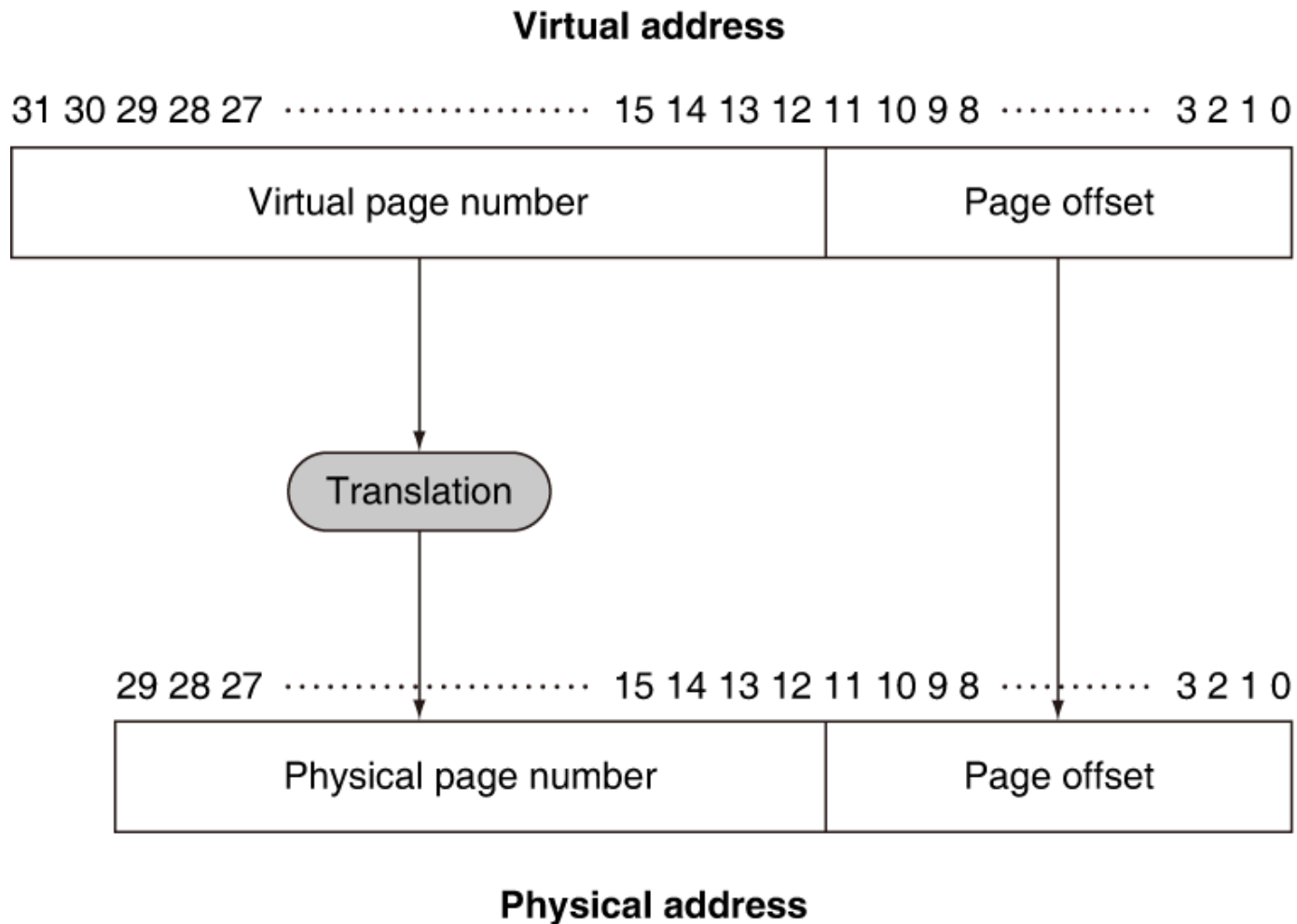


圖5.26 將虛擬位址對映到實體位址

頁大小為 $2^{12}=4$ KiB。由於實體頁編號有18 位元，在記憶體中所允許實體頁數為 2^{18} 頁。因此，主記憶體最多可到1 GiB，然而虛擬位址空間是4 GiB

5.7 虛擬記憶體

- 巨大的錯失懲罰導致設計虛擬記憶體系統時的許多關鍵決定：
 - 頁應該足夠大，以分攤這麼長的存取時間。目前典型的頁大小通常是從4 KiB 到16 KiB
 - 可以降低頁錯失率的設計是重要的。主要方法是，以全關聯放置法擺放記憶體中的頁
 - 降低頁錯失率可由軟體處理，因為和磁碟存取時間相比，軟體的處理成本是小的
 - 寫透方法不能用在虛擬記憶體中，因為寫入輔助記憶體花費太長的時間

放置一頁並且再找到這頁

- 若我們允許一個虛擬頁對映到任何的實體頁，則當頁錯失發生時，作業系統可以選擇任何的頁來置換
- 使用全關聯放置法的困難是找出一筆記錄
 - 我們使用一個索引記憶體的表來尋找頁；這個表稱為頁表 (page table)
 - 其存在(主)記憶體中
 - 頁表使用虛擬位址中的頁編號為索引值來找到相對應的實體頁編號
- 為了指出記憶體中頁表的位置，硬體中含有一個暫存器指到頁表的開始位置
 - 頁表暫存器

放置一頁並且再找到這頁

- 圖5.27 以頁表暫存器、一個虛擬位址以及所指定的頁表來表示硬體如何得出一個實體位址
 - 因為頁表包含了所有虛擬頁的對映關係，因此不需要標籤

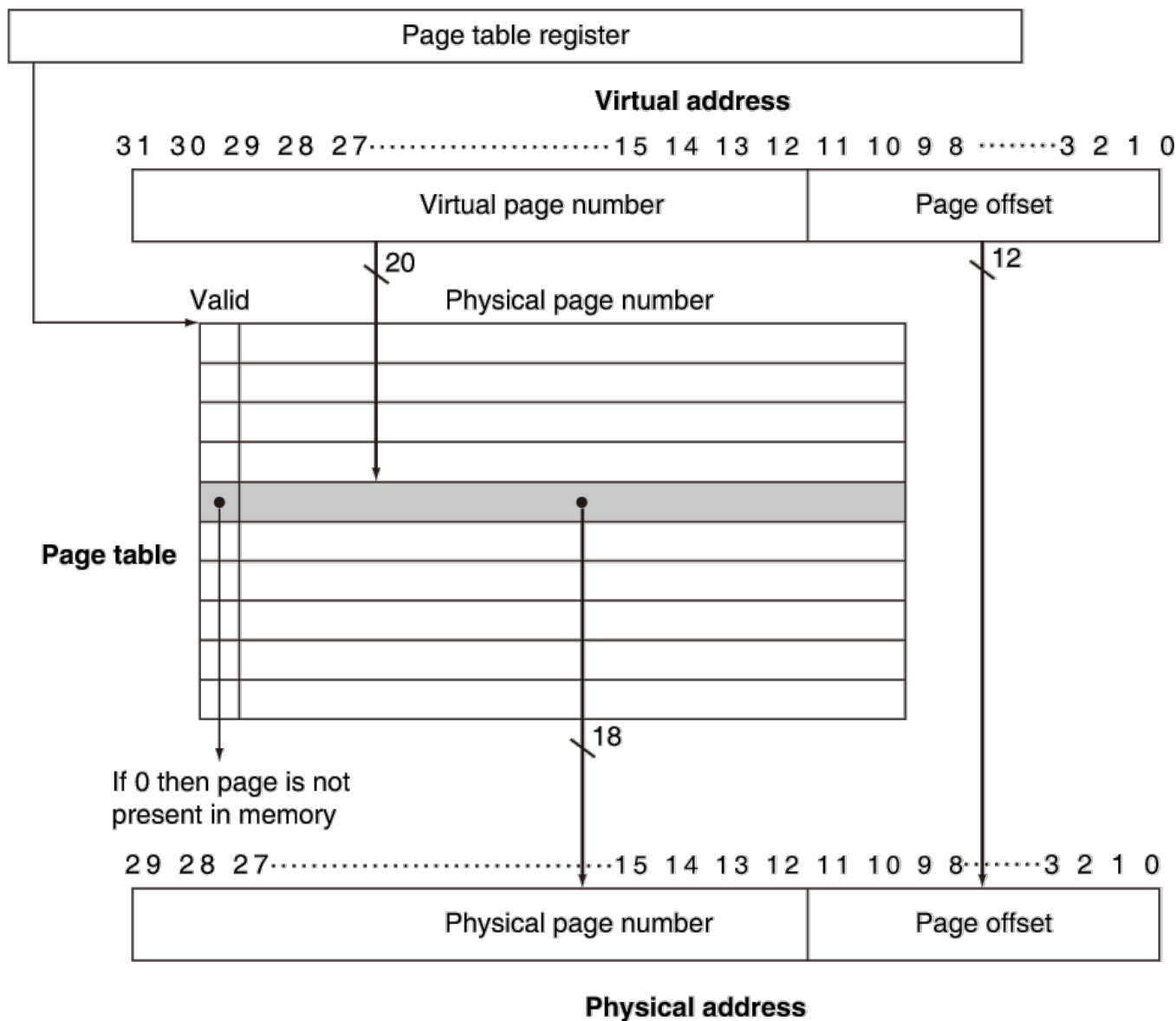


圖5.27 頁表以虛擬頁編號來索引以取得實體位址空間的對應部分。假設位址為**32 位元**

頁表的啟始位址由頁表指標指出。此圖中，頁大小為 **2^{12} 位元組**或**4 KiB**。虛擬位址空間是 **2^{32} 位元組**，或**4 GiB** 以及實體位址空間是 **2^{30} 位元組**，亦即主記憶體最多可到**1 GiB**。頁表中的記錄數為 **2^{20}** ，或一百萬筆記錄。每筆記錄中的有效位元指出該對映資料是否有效。若為否，則該頁不在主記憶體中。雖然這裡顯示的頁表記錄僅需**19 位元**，為了易於索引，它通常被擴大到**32 位元**。額外的位元可用來存放每頁需要保存的其他資訊，如保護級別等

頁錯失

- 若虛擬頁的有效位元為off，則發生頁錯失，必須由作業系統來處理
 - 其必須在階層中的下一層(通常是快閃記憶體或磁碟機)找到該頁，並且決定該頁應放置在主記憶體中的何處
- 磁碟中頁的位置無法僅由虛擬位址得知
 - 我們必須記錄在虛擬位址空間中的每一頁，其在磁碟中的位置
 - 圖5.28 表示了一個單一的存有實體頁編號或磁碟位址的表結構
- 作業系統也建立一個資料結構，來追蹤每一個實體頁被哪些程序和虛擬位址所使用

Virtual page number

Page table
Valid Physical page or disk address

1	
1	
1	
1	
0	
1	
1	
0	
1	
1	
0	
1	

Physical memory

Disk storage

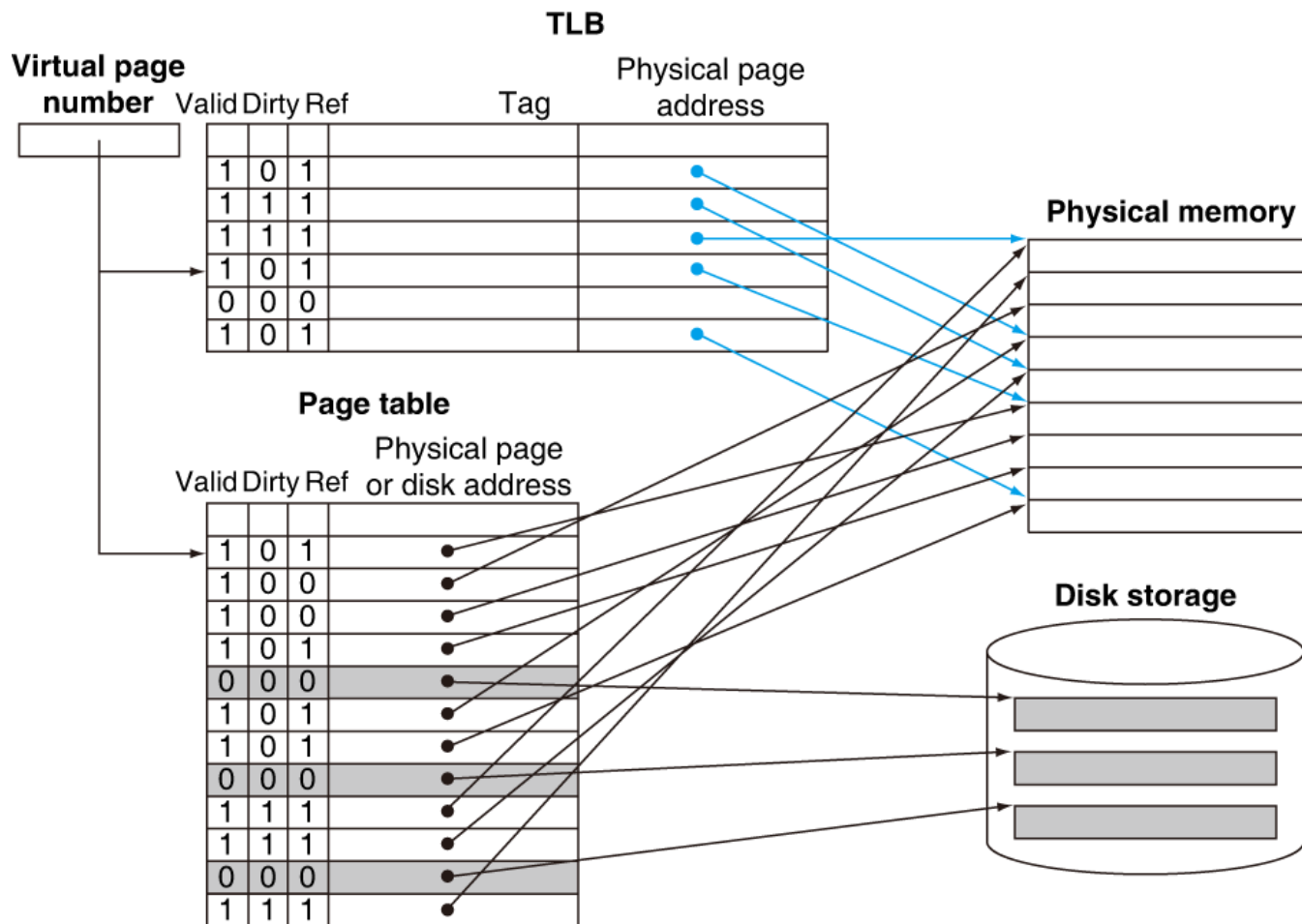
圖5.28 頁表將虛擬記憶體中的每一頁對映到主記憶體中的一頁，或可對映到存放在記憶體階層下一層的磁碟中儲存的一個頁。虛擬頁編號用來索引頁表。若有效位元為on，頁表提供相對應於虛擬頁的實體頁編號(即記憶體中頁的起始位址)。若有效位元為off，該頁目前只存在磁碟中某指定位址。在許多系統中，雖然存放實體頁位址與磁碟頁位址的表在邏輯上為同一份，但其存於二個分開的資料結構裡。使用二個表合理性，部分是由於即使有些頁目前在主記憶體中，我們仍必須保存所有頁的磁碟位址。記得，主記憶體中的頁和磁碟上的頁是相同大小的。

關於寫入呢？

- 在虛擬記憶體系統中，寫入階層中的下一層級(硬碟)需花費處理器數百萬計的週期數
 - 所以虛擬記憶體系統必須使用寫回方法

使位址轉換快速：TLB

- 因為頁表存放在主記憶體中，程式中每次的記憶體存取至少要花2 倍時間：第一次記憶體存取得到實體位址，第二次存取得到資料
- 近來的處理器使用了一個記錄最近用過的一些轉譯資料的特別快取
 - 轉譯側查緩衝器(translation-lookaside buffer, TLB)
- 圖5.29每次存取時，我們在TLB 中尋找該虛擬頁編號
 - 若TLB 中發生錯失，我們必須判斷此錯失為頁錯失或僅僅是TLB 的錯失
- 因為TLB 記錄中含有參照和污染位元，當我們置換其記錄時，需要將這些位元複製回頁表的記錄中



Dirty bit:該頁被修改過
Ref bit:該頁被參考過

圖5.29 TLB 作為頁表的快取，只存放對映到實體頁的記錄

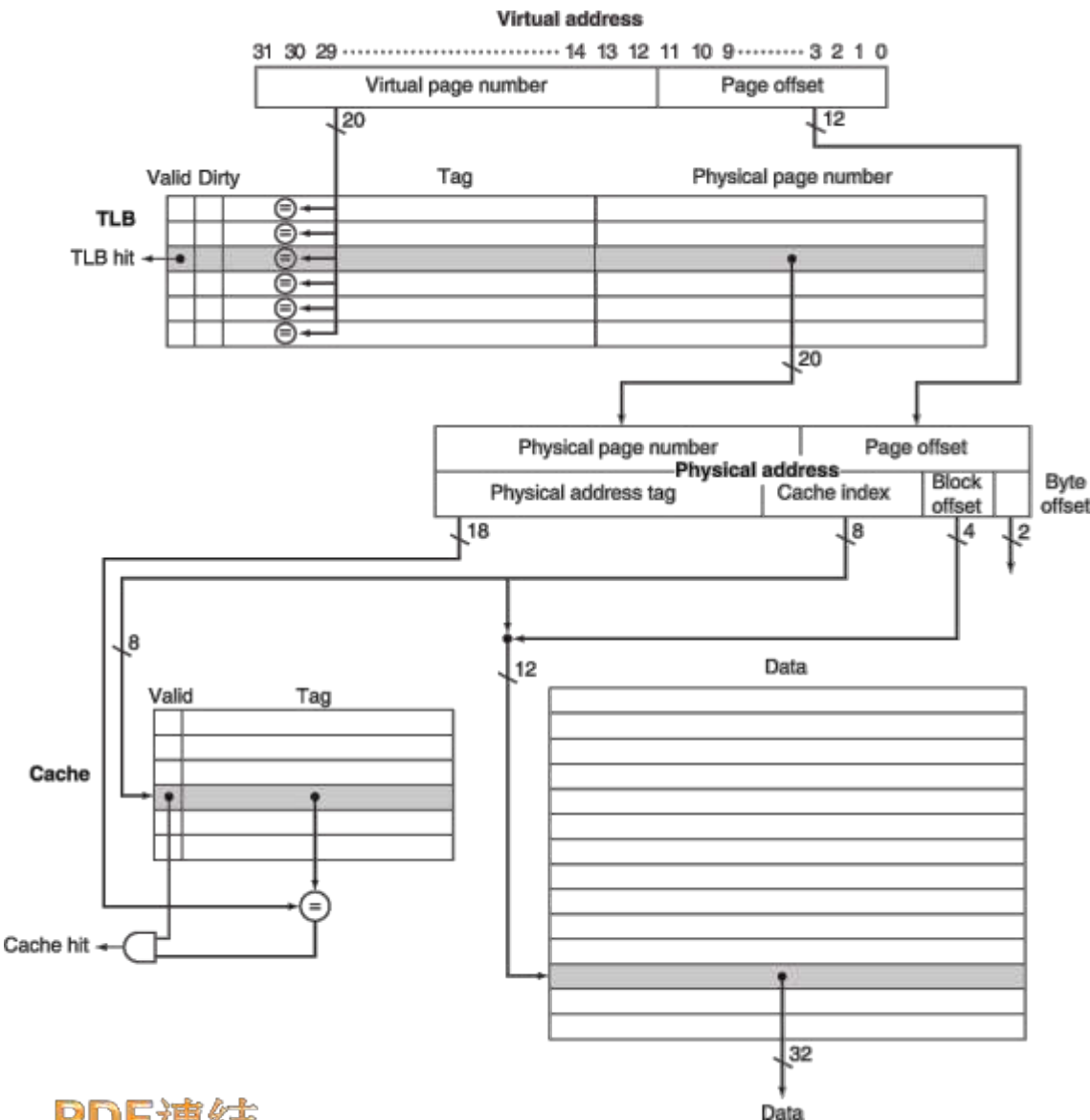
TLB 存放頁表中虛擬到實體頁對映資料的一部分。TLB 的對映資料以灰色表示。因為 TLB 是快取，它必需有標籤欄位。若一個頁在 TLB 中沒有符合的記錄，則必須查看頁表。頁表或者提供該頁的實體頁編號(其可用來建立一筆 TLB 記錄)，或者指出該頁在磁碟中，此情形為發生頁錯失。因為頁表有每一個虛擬頁的對映記錄，所以不需有標籤欄位；也就是說，不像 TLB，頁表不是一個快取

Intrinsity FastMATH TLB

- ▣ 其記憶體系統使用4 KiB 的頁和32 位元位址空間
- ▣ TLB 包含16 筆記錄，是全關聯式，並被指令和資料存取所共用
- ▣ 圖5.30 顯示TLB 和快取之一
- ▣ 圖5.31 顯示處理一個讀或寫請求時的步驟

圖5.30 Intrinsity FastMATH 中 TLB 與快取完成由虛擬位址到取得資料的過程

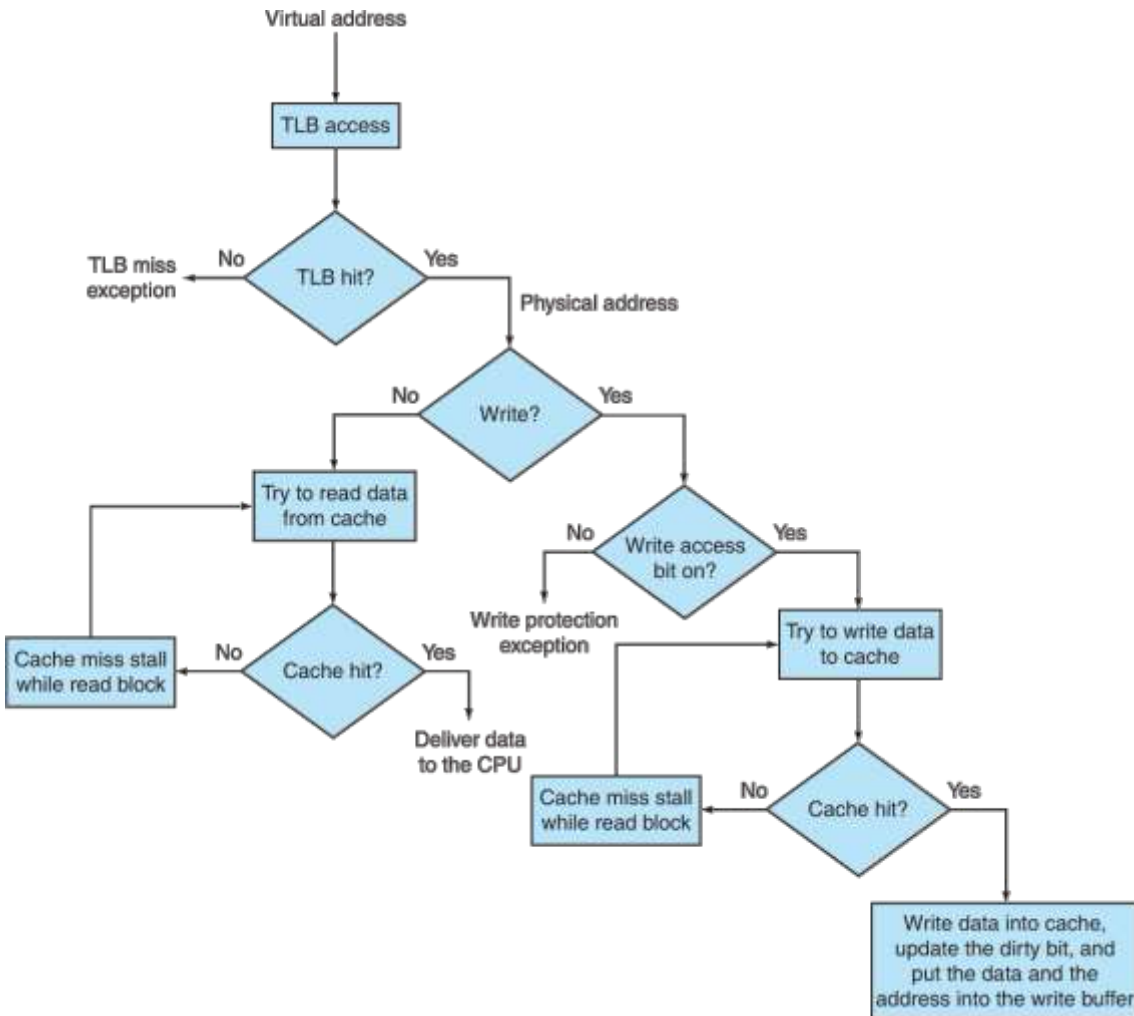
此圖顯示假設頁大小為4 KiB 時，TLB 和資料快取的組織。此圖強調讀取過程；圖5.31 則描述如何寫入。注意，不像圖5.12，標籤和資料RAM 是分開的。在長而窄的資料AM 中，以快取索引串接區塊偏移量來定址，我們不必使用16:1 多工器，即在區塊中選取所需的字組。在此快取是直接對映，而TLB 為全關聯式。設計全關聯式TLB，由於所需的記錄可能在TLB 的任何地方，因此需要將每一個TLB 標籤與虛擬頁編號做比較(見411 頁的仔細深思中的內容可定址記憶體)。若符合記錄的有效位元為on，為TLB 存取命中，則實體頁編號的位元與頁偏移的位元共同形成索引，以用來存取快取



PDF連結

圖5.31 在Intrinsity FastMATH
TLB 和快取中，處理讀取或寫透

若TLB 命中，可用其中得出的實體位址存取快取。讀取時，若快取命中則可提供資料，若錯失則在主記憶體傳回資料時引發停滯。若為寫入且命中時，當假設為寫透，則快取記錄的一部分會被覆蓋且該筆寫入的資料被送到寫入緩衝器。寫入錯失與讀取錯失並無不同，除了區塊在從主記憶體讀入之後，需被修改外。寫回要求寫入對該快取區塊設定污染位元，以及只有在讀取錯失或寫入錯失中，當被置換的區塊是污染時，才需將整個區塊載入寫入緩衝器。注意，TLB 命中和快取命中是獨立的事件，但是快取命中僅可發生在其代表資料一定已經存在主記憶體中的TLB 命中之後。在TLB 錯失和快取錯失之間的關係會在下述的例題以及本章最後的習題中進一步討論



記憶體階層的整體運作

在一個如圖5.30 所示，包含TLB 和快取的記憶體階層中，一個記憶體存取可能遇到三種錯失： TLB 錯失、頁錯失、和快取錯失。設想這三種錯失，有一種或多種發生的所有組合(七種可能性)。對每一種可能性，說明其是否真的會發生且在什麼情況下發生。

記憶體階層的整體運作

圖5.32 顯示所有組合和每一種組合實際上是否可能發生：

TLB	頁表	快取	是否可能？若可能，在什麼情況下發生？
命中	命中	錯失	可能，雖然當 TLB 命中時，就不會檢查頁表
錯失	命中	命中	TLB 錯失，但在頁表中找到記錄；重試之後，也在快取中找到資料
錯失	命中	錯失	TLB 錯失，但在頁表中找到記錄；重試之後，資料並不在快取中
錯失	錯失	錯失	TLB 錯失且之後頁錯失；重試之後，資料一定不在快取中
命中	錯失	錯失	不可能：若頁不在記憶體中，TLB 中不可能有轉譯的位址
命中	錯失	命中	不可能：若頁不在記憶體中，TLB 中不可能有轉譯的位址
錯失	錯失	命中	不可能：若頁不在記憶體中，資料不允許放在快取中

圖5.32 TLB、虛擬記憶體系統以及快取中可能事件的組合

有三種組合是不可能發生的以及有一種組合(TLB 命中、虛擬記憶體命中、快取錯失)是可能發生的但從不會偵測

- 圖5.32 假設所有記憶體位址已在快取存取之前被轉譯成實體位址
- 可以用全部或部分的虛擬位址來索引快取。此快取稱為虛擬定址快取(**virtually addressed cache**)
- 當快取以虛擬位址存取且頁是由多個程式共用(因此，可用不同的虛擬位址來存取他們)，有可能發生別名(**aliasing**)
- 相同的一頁有兩個虛擬位址，這種混淆使得一個該頁中的字組，因為可能被存放在快取的兩個不同位置，各自有其不同的虛擬位址而產生問題

- 一個常見的融合這兩種方法的做法是以虛擬位址來索引快取-甚至只使用位址中頁偏移量的部分，因為其不需被轉譯，所以這其實也就是實體位址-但使用實體標籤
 - 其目的是得到虛擬索引快取效能的好處以及實體定址快取(physically addressed cache)在架構上較簡單的好處

在虛擬記憶體中作出保護機制

- 如果作業系統保持各頁表中不相關的虛擬頁都對映到沒有重疊的(disjoint)實體頁，則一個程序將不可能存取其他程序的資料
 - 當然，這也需要使用者程序不能夠改變頁表對映
- 存取其他程序的資訊需要改變存取程序的頁表
 - 作業系統必須介入協助他們

- 當作業系統決定從執行程序P1 變成執行程序P2 時(稱為程序切換，context switch 或process switch)
 - 有TLB 時，我們必須清除屬於TLB 中關於P1 時的記錄
 - 常用的變通方法是藉由增加程序識別碼(process identifier) 或工作識別碼(task identifier)來擴充虛擬位址空間
 - 類似問題也會發生於快取中，因為在程序交換時，快取會包含執行中程序的資料
 - 這類問題對實體定址或虛擬定址的快取會以不同的形式發生。並有多種不同的解法

處理TLB 錯失和頁錯失

- ▣ TLB 錯失可以表示二種可能性之一
 1. 頁存在記憶體中，因此，我們僅需建立錯失的TLB 記錄
 2. 頁不存在記憶體中，因此，我們需要將控制轉換到作業系統去處理頁錯失
 - MIPS 一向以軟體處理TLB 錯失
- ▣ 為了在頁錯失處理後，要重新執行該指令，造成頁錯失指令的程式計數器必須被保存
 - 例外程式計數器(EPC)可用以保存此值

處理TLB 錯失和頁錯失

- TLB 錯失或頁錯失的例外必須在該記憶體存取週期結束前發出，以使下一個週期可開始例外的處理，而非繼續正常指令的執行
 - 若頁錯失不是在該週期發現，載入指令會覆寫(污染)暫存器
 - 類似的困難會發生在儲存指令上

硬體／軟體介面

- ▣ 從作業系統開始執行例外處理程序到它完成保留被插斷程序的所有狀態前，作業系統是特別脆弱的
- ▣ 可以藉由允許或不允許進一步例外致能(enable exception)的能力來避免慘劇

處理TLB 錯失和頁錯失

- 一旦作業系統知道造成頁錯失的虛擬位址時，必須完成三個步驟：
 1. 使用虛擬位址查閱頁表記錄來得知所要存取頁在磁碟中的位置
 2. 選擇要置換的實體頁；若被選取的頁是被修改過的，在我們將新的虛擬頁寫入該實體頁之前，該頁必須先被寫到磁碟中
 3. 開始從磁碟中讀入存取到的頁至選定的實體頁中
- 作業系統通常會在磁碟存取完成前，選擇另一程序放在處理器中來執行

處理TLB 錯失和頁錯失

- ▣ 在處理器中妥當地處理數據存取的頁錯失例外時，由於下列三種特性的同時存在而不易處理：
 1. 它們不像指令頁錯失，而是發生在指令執行過程的中間
 2. 在處理例外之前，此指令無法被完成
 3. 在處理例外之後，指令必須如同沒發生過例外般地重新執行

處理TLB 錯失和頁錯失

- 造成錯失的虛擬位址取決於該錯失是指令或數據錯失
 - 造成錯失的指令位址在EPC 中
 - 若此為指令頁錯失，則EPC 包含頁錯失的虛擬位址
 - 否則，可經由檢查指令(其位址在EPC 中)找到基底暫存器和偏移欄位算出錯失的虛擬位址

- 為了避免在低層級例外程式碼的頁錯失問題，MIPS 預留了不會發生頁錯失的位址空間，此稱為不做對映(unmapped)。
 - 作業系統將例外進入點的程式碼和例外堆疊放置在不做對映的記憶體中

- 為了避免在每一次記憶體存取中需要執行多一層的間接動作，VMM 使用了一個直接從客虛擬位址空間對應到實體位址空間、稱為影子頁表 (shadow page table) 的硬體
- 架構中最後一個要被虛擬化的部分是 I/O
 - 由於不斷增加的計算機 I/O 裝置數量以及 I/O 裝置類型上的更大歧異，這部分是系統虛擬化中最最困難的部分
 - 另一種困難是多個 VMs 共用一個真實裝置的情況

- 另一種是因為要支援為數極多的且均為必需的裝置驅動程式、特別是要在虛擬系統上支援許多不同的客OSes 時
- 對虛擬機器另一個難題是將其虛擬記憶體作虛擬化，因為在每一個虛擬機器中每一個客OS 都管理著他自己的一組頁表
 - 把真實記憶體當成是一個介於虛擬記憶體與實體記憶體之間的獨立的中間層級
 - 另外也有人以虛擬記憶體、實體記憶體和機器記憶體來分別稱呼這三個層級

總結

- ▣ 虛擬記憶體容許單一程式的位址空間超過主記憶體位址空間的限制。更重要地，虛擬記憶體支援多個同時執行中的程序在受保護的方式下共享主記憶體
- ▣ 有許多方法用來減少錯失率：
 1. 使用更大的頁
 2. 虛擬位址和實體位址間的對映用頁表來實現，並且做成全關聯式
 3. 作業系統使用如LRU 和存取位元的方法來選擇哪些頁被置換

總結

- ▣ 虛擬記憶體使用寫回方法，而且避免將未更改的頁寫回磁碟機中
- ▣ 位址轉換使得共享的主記憶體受到保護，並且提供許多額外的好處，例如簡化記憶體配置
- ▣ TLB 有如頁表轉譯的快取

5.8 記憶體階層的常用架構

- 雖然記憶體階層在很多方面的參數有所不同，但決定一個階層如何運作的許多策略與特性性質上是相似的
- 我們會以用於記憶體階層中任何兩層間一系列的四個問題來檢視這些策略；基於簡化問題，我們主要使用快取的術語

問題1：區塊可放置在哪裡？

- ▣ 直接對映、集合關聯式到全關聯式
 - 全部的方法都可被想為集合關聯式的變型
 - 增加關聯度的好處通常是會減少錯失率

問題2：如何找到一個區塊？

▣ 這些方法總結如下：

關聯度	定位方法	需要的比較數
直接對映	索引	1
集合關聯式	先索引到集合，搜尋集合中的資料	關聯度
全關聯式	搜尋所有的快取記錄	快取大小
	獨立的查表	0

問題2：如何找到一個區塊？

- 在任何記憶體階層中，在直接對映、集合關聯式或全關聯式之間的選擇，取決於錯失成本和實作關聯度時包含時間與額外硬體的成本
 - 在虛擬記憶體系統中，另有一塊對映表(即頁表)來索引記憶體。除了需要儲放表格的空間外，使用索引表也需要額外的記憶體存取

問題3：在快取錯失時，哪個區塊該被置換？

- 在集合關聯式或全關聯式快取中，有兩種主要的置換方法：
 - 隨機(random)
 - 最久沒用到(least recently used, LRU)

問題4：寫入時發生什麼事？

▣ 兩種基本的選擇

- 寫透(write-through)
- 寫回(write-back)
- 在虛擬記憶體系統中，只有寫回方法是可行的

3C：瞭解記憶體階層行為的直觀模型

- 在此模型中，所有的錯失被分為以下三類之一(3C，three Cs)：
 - 必須(發生)的錯失(compulsory misses)：
由第一次存取一個從未存在於快取中的區塊所造成。也稱為冷啟動錯失 (cold-start misses)
 - 容量錯失(capacity misses)
 - 衝突錯失(conflict misses)：
是在直接對映或集合關聯快取中會發生、但在同樣大小的全關聯快取中，卻不會發生的錯失。也稱做碰撞錯失(collision misses)
- 圖5.37 顯示錯失率如何被分成三個來源

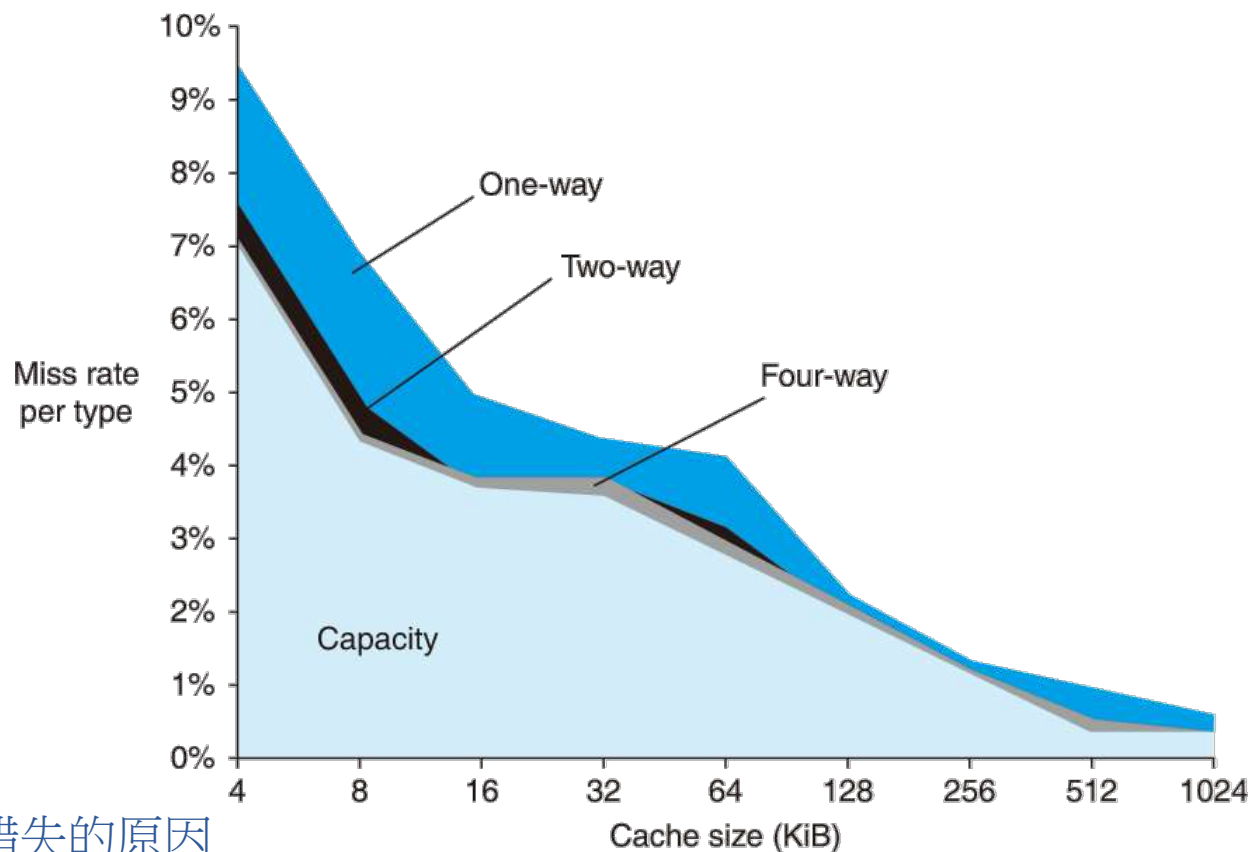


圖5.37 錯失率來自三種錯失的原因

此圖顯示不同大小的快取的總錯失率和錯失成份。該數據由SPEC CPU2000 整數和浮點數測試基準程式取得且與圖5.36 中的資料的來源相同。必須(發生)的錯失成份是0.006%，無法在圖中顯示。下一個成份是容量錯失率，其與快取大小有關。衝突錯失與關聯度和快取大小均有關，並由從一路到八路的關聯度表示之。每一種情況，標示的區域即為，關聯度從其上一個關聯度變成該標示的關聯度間錯失率的增加。例如，標示二路的區域指出當快取有二路而不是四路關聯度時所引起額外的錯失。因此，在相同大小因直接對映與全關聯式的不同，而導致錯失率的差異則由標示八路、四路、二路以及一路區域的總合所給定。八路和四路間的差異非常小，很難在圖中看出

5.9 以有限狀態機來控制簡單快取

一個簡單的快取

▣ 快取的主要特性：

- 直接對映快取
- 寫回並使用寫入配置
- 區塊大小是4 個字組(16 位元組或128 位元)
- 快取大小是16 KiB，所以它有1024 個區塊
- 32 位元的位元組位址
- 快取中的每個區塊包含有效位元和污染位元

一個簡單的快取

▣ 位址中與快取相關的各欄位：

- 快取索引有10 個位元
- 區塊偏移量是4 個位元
- 標籤大小是 $32 - (10 + 4)$ 或18 個位元

一個簡單的快取

- ▣ 處理器和快取間的信號是：
 - 1 位元的讀或寫信號
 - 1 位元的有效信號，表示是否有快取運作
 - 32 位元的位址
 - 從處理器到快取的32 位元資料
 - 從快取到處理器的32 位元資料
 - 1 位元的備妥信號，表示快取運作是否完成

一個簡單的快取

- 記憶體和快取之間的介面信號是：
 - 1 位元讀取或寫入信號
 - 1 位元有效信號，表示是否有記憶體運作
 - 32 位元位址
 - 從快取到記憶體的128 位元資料
 - 從記憶體到快取的128 位元資料
 - 1 位元的備妥信號，表示記憶體運作是否完成

有限狀態機

- ▣ 快取因為運作可以是一連串的步骤而使得控制更複雜
- ▣ 有限狀態機包含一組狀態以及如何改變狀態的說明
 - 當我們將有限狀態機用於控制，每一狀態也指明一組當機器處於該狀態時要被設定的輸出
 - 在有限狀態機中，我們對所有有關的多工器控制總會指明其設定
- ▣ 圖5.39 是此實作可能的樣子

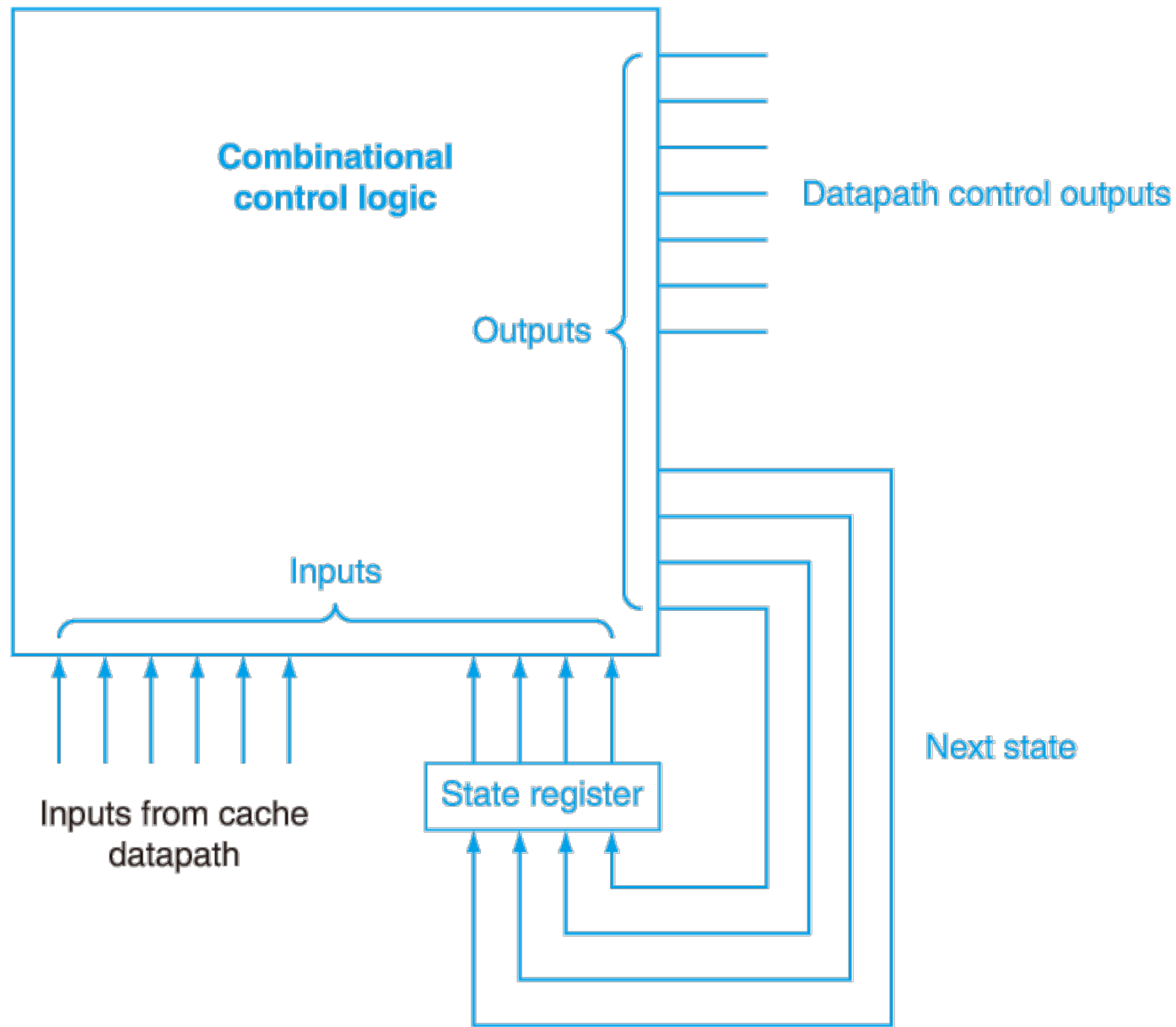
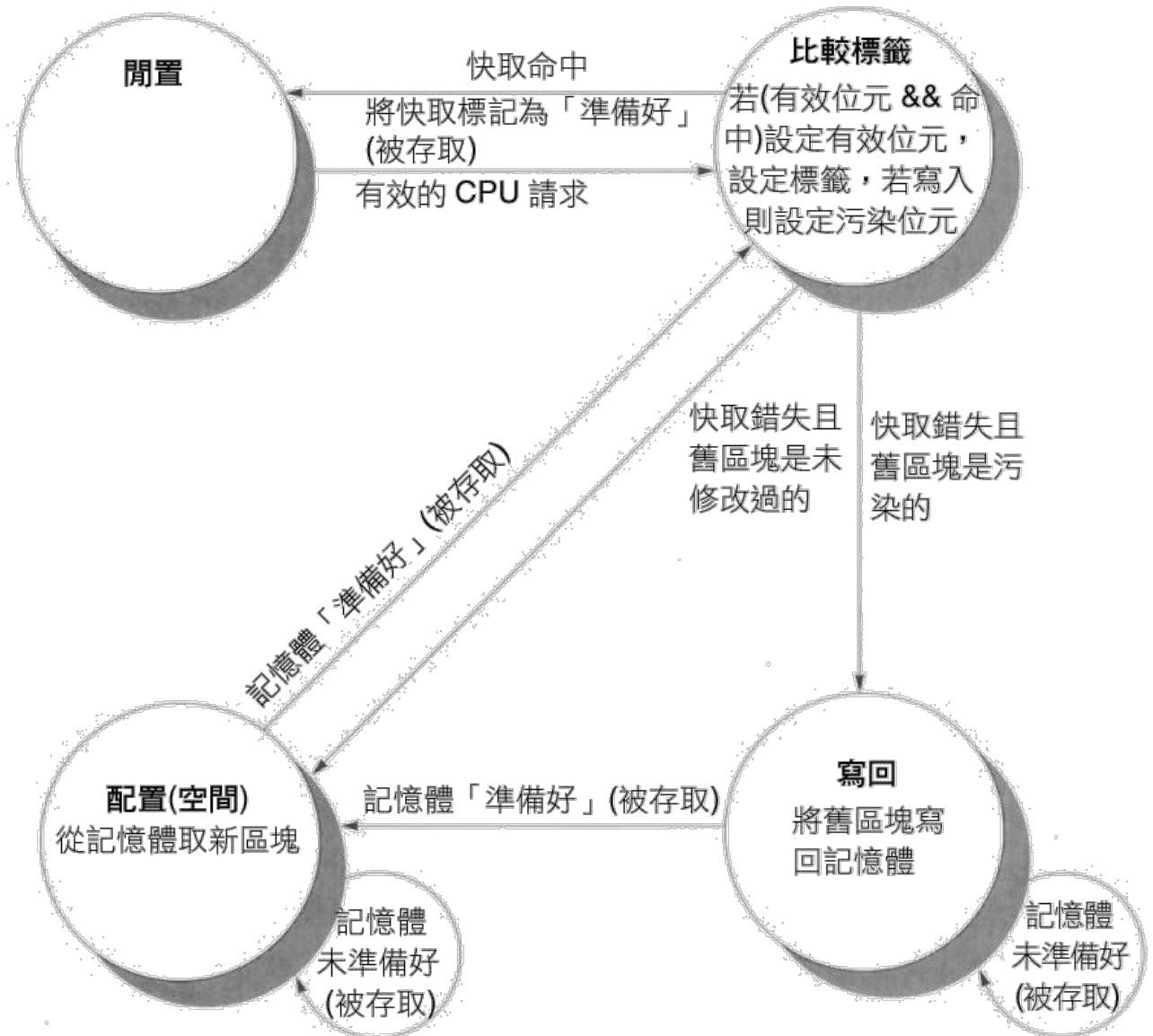


圖5.39 有限狀態機控制器通常使用組合邏輯方塊以及一個暫存器來實作目前狀態的保存。組合邏輯的輸出是下一狀態的號碼以及目前狀態能被 **assert** 的控制信號。輸入到組合邏輯的是目前狀態以及用來決定下一狀態的任何輸入。在本例中，輸入是指令暫存器運算碼位元。注意，本章使用的有限狀態機中，輸出僅取決於目前狀態，而非輸入。仔細深思有更詳細的解釋。

- ▣ 阻斷式的快取(blocking cache)
- ▣ 非阻斷式的快取(nonblocking cache)
- ▣ Moore 機器(Moore machine)特徵是輸出僅與目前狀態有關
- ▣ Mealy 機器容許同時用輸入和目前狀態來決定輸出
 - Moore 機器在實作上有速度和控制單元大小的潛在優勢。潛在的劣勢是可能需要額外的狀態

簡單快取控制器的FSM

- 圖5.40 顯示我們的簡單快取控制器的四個狀態：



簡單快取控制器的FSM

- 此簡單模式可輕易地擴展成更多狀態，以求改善效能
 - 例如，比較標籤狀態在單一時脈週期內，同時處理了比較和讀取或寫入快取資料
 - 增加一個寫入緩衝器

5.10 平行性與記憶體階層：快取一致性

- 如果一個多核的多處理器意謂著多個處理器在單一晶片上，則這些處理器很可能共用一個共同的實體位址空間
- 透過快取來存取這些共用的資料引發一個新的問題：
由於兩個不同處理器是透過他們各自的快取來看記憶體，因此若沒有額外的事先預防措施，結果對共用的數據可能會看到不同的值
 - 圖5.41 說明快取一致性問題(cache coherence problem)

時間 步驟	事件	CPU A 的 快取內容	CPU B 的 快取內容	位置X 的記 憶體內容
0				0
1	CPU A 讀取X	0		0
2	CPU B 讀取X	0	0	0
3	CPU A 將1 存入X	1	0	1

圖5.41 對於單一記憶體位置(X)、被二個處理器(A和B)讀取和寫入時的快取一致性問題

我們假設快取開始時皆不包含這個變數且X的值為0。我們也假設此是寫透式的快取；寫回快取會增加上一些額外但類似的複雜性。在X的值被A寫入之後，A的快取和記憶體皆包含新值，B的快取則無新值，且若B讀取X值時，則會得到0！

5.10 平行性與記憶體階層：快取一致性

- 記憶體系統行為中兩個不同的面向，兩者對撰寫正確的共用記憶體程式，都非常重要
 - 第一個面向稱為一致性(**coherence**)，定義讀取時什麼值可以被傳回
 - 第二個面向稱為一貫性(**consistency**)，決定寫入的值何時將被傳回至其他的讀取

強制一致性的基本方法

- ▣ 在快取一致性的多處理器中，快取提供共用資料項目的遷移(migration) 和複製(replication)
 - 支援遷移和複製對於存取共用資料的效能是重要的，所以很多的多處理器採用硬體協定來維護具一致性的快取
- ▣ 快取一致性協定(cache coherence protocols)
 - 窺探(snooping)
 - 每一個快取在擁有一份實體記憶體區塊的資料時，同時也擁有一份區塊的共用狀態，而該等狀態並不作集中保存

強制一致性的基本方法

- 所有的快取都可以經由某種廣播媒介(匯流排或網路)來存取，所有的快取控制器也都透過這個媒介來監視或窺探以得知他們是否擁有正在匯流排上被存取的資料區塊或應將存取導向別的快取

窺探協定

- 強制一致性的一個方法是確保處理器在對一個資料項目寫入前，取得排他性的存取權
 - 寫入作廢協定(write invalidate protocol)
 - 圖5.42 說明使用寫回快取時在其窺探匯流排上作廢協定的一個例子
 - 此協定也強制了寫入的循序化

處理器動作	匯流排動作	CPU A 的 快取內容	CPU B 的 快取內容	位置X 的記 憶體內容
				0
CPU A 讀取X	快取對X 的錯失	0		0
CPU B 讀取X	快取對X 的錯失	0	0	0
CPU A 將1 寫 入X	X 失效	1		1
CPU B 讀取X	快取對X 的錯失	1	1	1

圖5.42 使無效協定對於寫回式快取中的單一快取區塊，運作於窺探式匯流排上的例子

我們假設開始時快取都不包含X 及X 在記憶體中的值為0。CPU 和記憶體內容欄位表示的是處理器與匯流排動作均完成後的值。空白表示沒有動作或是或沒有副本被置入快取。當由B 引發的第二次錯失發生時，CPU A 回以該值並阻擋了記憶體的回應。同時，X 在B 快取及記憶體的內容也都作了更新。這種當區塊變成共享時造成的記憶體隨之更新簡化了協定，然而亦可記住該區塊的擁有者而僅在其所擁有的該區塊要被置換時才作寫回。這方法需要一個稱為「擁有者」的額外狀態，用以指出一區塊可能被共享，不過擁有它的處理器當負責在改變其值或將其置換時，要適當地通知其他處理器及記憶體

硬體／軟體介面

- ▣ 假共用(false sharing)：當兩個不相關的共用變數處於相同的快取區塊中，即使多個處理器正存取不同的變數，也必須在多個處理器間以整個區塊來傳遞
 - 程式師和編譯器應該要小心擺放資料以避免假共用發生

- 一個寫入的資料確實在何時可被讀取者看到的問題是由記憶體一貫性模型來定義
 - 我們做下列兩個假設
 - 首先，一個寫入需等到所有處理器都看到寫入的結果才算完成(並允許下一個寫入開始)
 - 第二，處理器不改變任何寫入相對於其他記憶體存取的順序

- 因為輸入會改變在快取之後的記憶體、以及因為輸出需要用到存在寫回快取裡的最新的值，所以分別有一種在處理器裡存在有多層快取時、以及另一種只是在多個處理器的快取間的I/O的快取一致性問題
- 另有目錄式(directory-based)快取一致性協定只在一個稱為目錄 (directory)的位置保存實體記憶體區塊的共用狀態。
 - 目錄式快取一致性比窺探式有稍高一點的製作成本，但它可減少快取間的交通量，並且因此可以用在更大規模的處理器數量上

5.13 實例：ARM Cortex-A8 與Intel Core i7的記憶體階層

■ 圖5.43 概述這兩個處理器的位址空間與TLBs

- A8 有兩個TLBs 以及32 位元的虛擬位址空間與32 位元的實體位址空間
- Core i7 有三個TLBs 以及48 位元的虛擬位址與44 位元的實體位址

特性	ARM Cortex-A8	Inter Core i7
虛擬位址	32 位元	48 位元
實體位址	32 位元	44 位元
頁大小	可調：4, 16, 64 KiB, 1, 16 MiB	可調：4 Kib, 2/4 MiB
TLB 組織	一個指令 TLB 與一個數據 TLB	每核一個指令 TLB 與一個數據 TLB
	兩個 TLBs 都是全關聯式，32 項次，循環置換	兩個 L1 TLBs 都是回路集合關聯式，LRU 置換
	TLB 錯失以硬體處理	L1 I-TLB 對小的頁有 128 項次，對大的頁每緒 7 項次
		L1 D-TLB 對小的頁有 64 項次，對大的頁有 32 項次
		L2 TLB 是四路集合關聯式，LRU 置換
		L2 TLB 有 512 項次
		TLB 錯失以硬體處理

圖5.43 ARM Cortex-A8 與Intel Core i7 920 的位址轉換與TLB 硬體

二個處理器都提供對大的頁(可用於像是作業系統中或是對映到畫面緩衝器 **frame buffer** 時)的支援。採用大型頁的方式可以避免對單一個總是會用到的物件卻需要用到大量的項次來對映它

5.13 實例：ARM Cortex-A8 與Intel Core i7的記憶體階層

▣ 圖5.44 說明它們之中的快取

特性	ARM Cortex-A8	Intel Nehalem
L1 快取組織	分開式的指令與數據快取	分開的指令與數據快取
L1 快取大小	指令 / 數據各有 32 KiB	每核指令 / 數據各有 32 KiB
L1 快取關聯度	4 路(I)，4 路(D)集合關聯式	4 路(I)，8 路(D)集合關聯式
L1 置換方法	隨機	近似的 LRU
L1 區塊大小	64 位元組	64 位元組
L1 寫入策略	寫回，寫一配置(?)	寫回，無寫一配置
L1 命中時間(載入—使用)	1 時脈週期	4 時脈週期，管道化

圖5.44 在ARM Cortex-A8 與Intel Core i7 920 中的各個快取

特性	ARM Cortex-A8	Intel Nehalem
L2 快取組織	合併式(指令與數據)	每核合併式(指令與數據)
L2 快取大小	128 KiB 至 1 MiB	256 KiB (0.25 MiB)
L2 快取關聯度	8-路集合關聯式	8 路集合關聯式
L2 置換方法	隨機(?)	近似的 LRU
L2 區塊大小	64 位元組	64 位元組
L2 寫入策略	寫回，寫一配置(?)	寫回，寫配置
L2 命中時間	11 時脈週期	10 時脈週期
L3 快取組織	—	合併式(指令與數據)
L3 快取大小	—	8 MiB，共用
L3 快取關聯度	—	16 路集合關聯式
L3 置換方法	—	近似的 LRU
L3 區塊大小	—	64 位元組
L3 寫入策略	—	寫回，寫配置
L3 命中時間	—	35 時脈週期

5.13 實例：ARM Cortex-A8 與Intel Core i7的記憶體階層

- 如何支援如A8 與Core i7 般可以在一個時脈週期裡執行多於一道記憶體指令的處理器？
 - 一種常用的方法是將快取分成多個排(banks)
- Core i7 具有可以降低錯失懲罰的額外最佳化
 - 在快取錯失時會將區塊中被需求的字組最先送回快取
 - 快取在快取錯失時同時也可繼續執行存取數據快取的指令，非阻斷式快取(nonblocking cache)
 - 錯失之下的命中(hit under miss)
 - 錯失之下的錯失(miss under miss)
- Core i7 對數據存取有一個預取的機制

5.14 走得更快：快取區塊分割與矩陣乘法

- 圖5.48 列出圖4.80 DGEMM 程式經區塊分割後的版本
 - 用第四章中DGEMM 未作迴圈展開的版本作為開始，並且對A、B 及C 中的部分矩陣呼叫它許多次

```

1 #include <x86intrin.h>
2 #define UNROLL (4)
3 #define BLOCKSIZE 32
4 void do_block (int n, int si, int sj, int sk,
5               double *A, double *B, double *C)
6 {
7     for ( int i = si; i < si+BLOCKSIZE; i+=UNROLL*4 )
8         for ( int j = sj; j < sj+BLOCKSIZE; j++ ) {
9             __m256d c[4];
10            for ( int x = 0; x < UNROLL; x++ )
11                c[x] = _mm256_load_pd(C+i+x*4+j*n);
12            /* c[x] = C[i][j] */
13            for( int k = sk; k < sk+BLOCKSIZE; k++ )
14            {
15                __m256d b = _mm256_broadcast_sd(B+k+j*n);
16                /* b = B[k][j] */
17                for (int x = 0; x < UNROLL; x++)
18                    c[x] = _mm256_add_pd(c[x], /* c[x]+=A[i][k]*b */
19                                       _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
20            }
21
22
23            for ( int x = 0; x < UNROLL; x++ )
24                _mm256_store_pd(C+i+x*4+j*n, c[x]);
25            /* C[i][j] = c[x] */
26        }
27
28 void dgemm (int n, double* A, double* B, double* C)
29 {
30     for ( int sj = 0; sj < n; sj += BLOCKSIZE )
31         for ( int si = 0; si < n; si += BLOCKSIZE )
32             for ( int sk = 0; sk < n; sk += BLOCKSIZE )
33                 do_block(n, si, sj, sk, A, B, C);
34 }

```

圖5.48 圖4.80 中的經過快取 區塊分割最佳 化的DGEMM C 程式

這些改變與在
圖5.21 中所用
的方法相同。
編譯器對
do_block 函數
所產出的組合
語言(碼)幾乎與
圖4.81 中所見
的完全相同。
再次強調，呼
叫**do_block** 並
不會有額外的
負擔，因為編
譯器已經將函
數呼叫改作了
嵌入(**inline**)

PDF連結

5.14 走得更快：快取區塊分割與矩陣乘法

- ▣ 圖5.49 比較了未最佳化的結果以及加上部分字平行性、指令階層平行性和快取相關等最佳化的結果

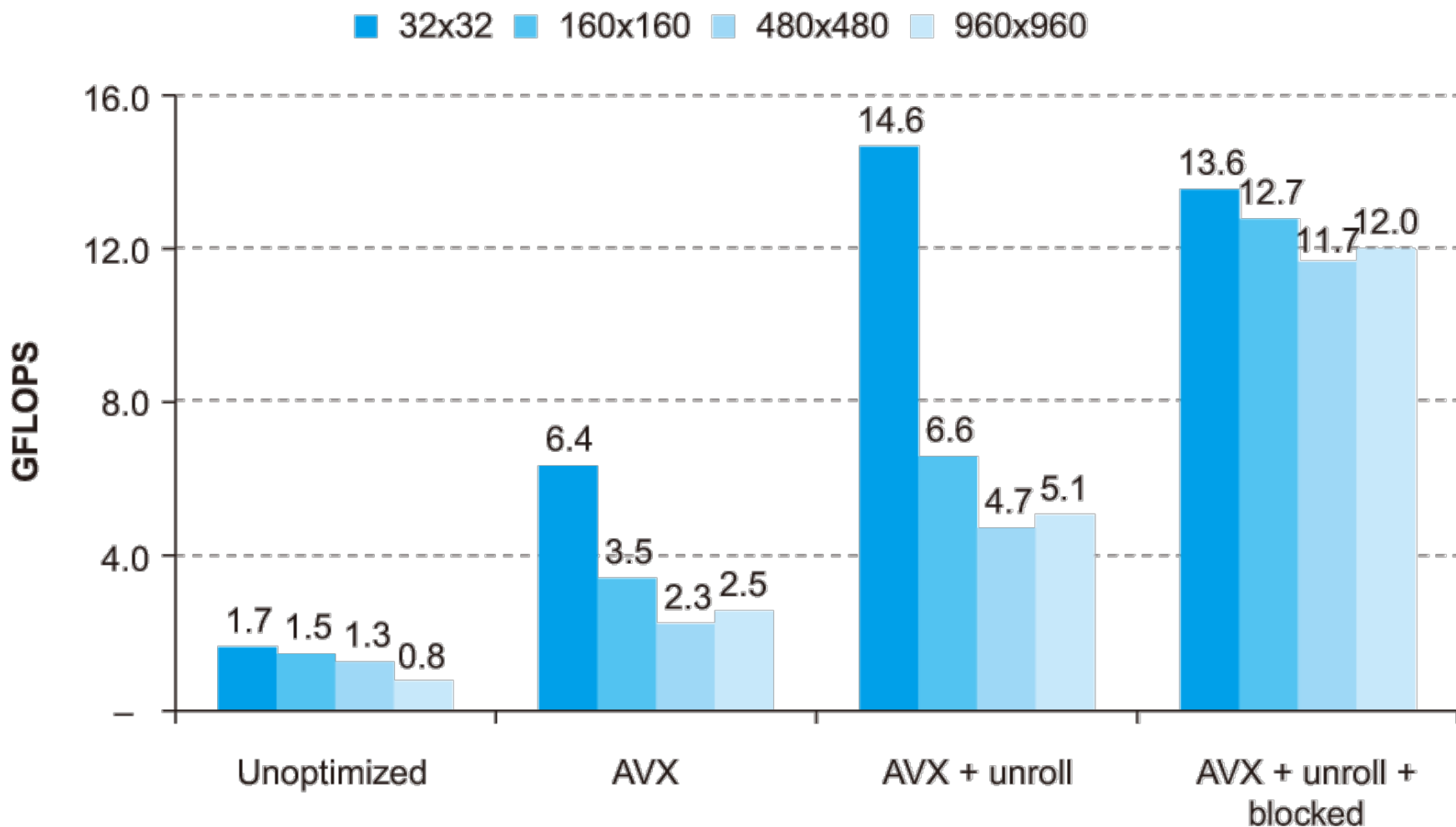


圖5.49 當矩陣大小由32x32 逐漸變成960x960 時，四種版本的DGEMM的效能
充份最佳化後的程式對最大的矩陣大小幾乎具有在第三章圖3.21 中未經最佳化的版本的15 倍快

5.15 謬誤與陷阱

- ▣ 記憶體階層是計算機架構中最適宜量化的面向之一，它也似乎比較不易受謬誤與陷阱所害
- ▣ 然而不僅有許多的謬誤已散播以及陷阱也被誤觸，而且有些還造成重大的負面結果
- ▣ 陷阱：在模擬快取時，忘記考慮位元組位址或是快取區塊大小
- ▣ 陷阱：在模擬快取的時候忘記考慮位元組定址的背景或者快取區塊的大小
- ▣ 陷阱：共用快取中只用低於共用快取的核(**cores**)或線(**threads**)數目的集合關聯度

5.15 謬誤與陷阱

- ▣ 陷阱：以平均記憶體存取時間來評估亂序執行處理器的記憶體階層表現
- ▣ 陷阱：藉由在未分段的位址空間上層加入段來延展位址空間
- ▣ 謬誤：磁碟在工作場所中的故障率與其規格所述者符合
- ▣ 謬誤：作業系統是處理磁碟存取排程最好的地方
 - 只有磁碟本身才會知道由邏輯位址到實體扇區、軌道以及表面之間的對應關係，它本身才知道應該如何透過重新排程來降低旋轉與尋找所造成的延遲

5.15 謬誤與陷阱

- ▣ 陷阱：在未被設計為可虛擬化的指令集架構上實作虛擬機器的監控器

5.16 總結評論

- ▣ 區域性原則
- ▣ 多層級快取因為兩個原因使得更多的快取最佳化方法更易於實現
- ▣ 另一個趨勢是尋求軟體的幫助
 - 利用程式轉換與硬體機制來有效地管理記憶體階層
- ▣ 預先提取(prefetching)
- ▣ 使用特別的具快取意識(cache-aware)的指令